

Entropy

Entropy

- The entropy of a discrete RV X w/ pmf P_X is

$$H(X) = -\sum_x P_X(x) \log \frac{1}{P_X(x)}$$

✓ For x s.t. $P_X(x) = 0$, we have $H(x) = 0 \log \frac{1}{0} = 0$

- Entropy $H(X)$ is the uncertainty associated w/ the RV X .

- Let X be a discrete RV taking values in $\mathcal{X}$. Denoting the alphabet size $|\mathcal{X}|$ by M ,

$$\textcircled{1} H(X) \geq 0$$

$$\forall x \in \mathcal{X}, P_X(x) \in [0, 1] \rightarrow \frac{1}{P_X(x)} \geq 1 \rightarrow \log \frac{1}{P_X(x)} \geq 0$$

$$\therefore H(X) = -\sum_x P_X(x) \log \frac{1}{P_X(x)} \geq 0$$

* $H(X) = 0$ when $P_X(x) = 0$ or $P_X(x) = 1$

$$\textcircled{2} H(X) \leq \log M$$

$$H(X) - \log M = -\sum_x P_X(x) \log \frac{1}{P_X(x)} - \sum_x P_X(x) \log M = -\frac{1}{\ln 2} \sum_x P_X(x) \ln \frac{1}{M P_X(x)}$$

$$\ln x \leq x - 1 \leq \frac{1}{\ln 2} \sum_x P_X(x) \left(\frac{1}{M P_X(x)} - 1 \right) = \frac{1}{\ln 2} \left(\sum_x \frac{1}{M} - \sum_x P_X(x) \right) = 0 \rightarrow H(X) \leq \log M$$

\textcircled{3} The equiprobable distribution $(\frac{1}{M}, \frac{1}{M}, \dots, \frac{1}{M})$ has max. entropy, equal to $\log M$.

The inequality $\ln x \leq x - 1$ is an equality when $x = 1$, so entropy is max. when

$$\frac{1}{M P_X(x)} = 1 \rightarrow P_X(x) = \frac{1}{M} \quad \forall x$$

- For $X \sim \text{Ber}(p)$, the entropy $H(X)$ is given by the binary entropy function $H_2(p)$.

$$H_2(p) = p \log \frac{1}{p} + (1-p) \log \frac{1}{1-p}$$

Joint and conditional entropy

- the joint entropy of discrete RVs X, Y w/ joint pmf $P_{X,Y}$ is

$$H(X, Y) = -\sum_{x,y} P_{X,Y}(x,y) \log \frac{1}{P_{X,Y}(x,y)}$$

- The conditional entropy of Y given X is the uncertainty in Y given a particular x , averaged over x .

$$H(Y|X) = E[H(Y|X=x)] = \sum_x P_X(x) H(Y|X=x)$$

$$= \sum_x P_X(x) \sum_y P_{Y|X}(y|x) \log \frac{1}{P_{Y|X}(y|x)} = \sum_{x,y} P_X(x) P_{Y|X}(y|x) \log \frac{1}{P_{Y|X}(y|x)}$$

$$\therefore H(Y|X) = \sum_{x,y} P_{X,Y}(x,y) \log \frac{1}{P_{Y|X}(y|x)}$$

- The joint entropy $H(X, Y)$ can be expanded: $H(X, Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$

$$H(X, Y) = -\sum_{x,y} P_{X,Y}(x,y) \log \frac{1}{P_{X,Y}(x,y)} = \sum_x \log \frac{1}{P_X(x)} \sum_y P_{Y|X}(y|x) + \sum_{x,y} P_{X,Y}(x,y) \log \frac{1}{P_{Y|X}(y|x)} = H(X) + H(Y|X)$$

$$= \sum_y \left(\log \frac{1}{P_Y(y)} \sum_x P_{X|Y}(x|y) \right) + \sum_{x,y} P_{X,Y}(x,y) \log \frac{1}{P_{X|Y}(x|y)} = H(Y) + H(X|Y)$$

- For independent X, Y , $P_{Y|X}(y|x) = P_Y(y) \rightarrow H(Y|X) = H(Y)$, $P_{X|Y}(x|y) = P_X(x) \rightarrow H(X|Y) = H(X)$

$$\therefore H(X, Y) = H(X) + H(Y)$$

- For $Y=f(X)$, $P_{Y|X}(y|x) = 1 \rightarrow H(Y|X) = 0$

If f^{-1} exists, then $P_{X|Y}(x|y) = 1 \rightarrow H(X|Y) = 0$

For Personal Use Only -bkwk2

Joint and conditional entropy for multiple RVs

- The joint entropy of $X_1 \dots X_n$ w/ joint pmf $P_{X_1 \dots X_n}$ is

$$H(X_1 \dots X_n) = - \sum_{x_1 \dots x_n} P_{X_1 \dots X_n}(x_1 \dots x_n) \log \frac{1}{P_{X_1 \dots X_n}(x_1 \dots x_n)}$$

- The conditional entropy of X_i given $X_1, X_2 \dots X_{i-1}$ is

$$H(X_i | X_1 \dots X_{i-1}) = - \sum_{x_1 \dots x_{i-1}} P_{X_1 \dots X_{i-1}}(x_1 \dots x_{i-1}) \log \frac{1}{P_{X_i | X_1 \dots X_{i-1}}(x_i | x_1 \dots x_{i-1})}$$

- In general, the joint entropy can be decomposed using the chain rule

$$H(X_1 \dots X_n) = H(X_1) + H(X_2 | X_1) + \dots + H(X_n | X_1 \dots X_{n-1}) = \sum_{i=1}^n H(X_i | X_1 \dots X_{i-1})$$

$$\begin{aligned} \hookrightarrow H(X_1 \dots X_n) &= - \sum_{x_1 \dots x_n} P_{X_1 \dots X_n}(x_1 \dots x_n) \log P_{X_1 \dots X_n}(x_1 \dots x_n) = - \sum_{x_1 \dots x_n} P_{X_1 \dots X_n}(x_1 \dots x_n) \log \prod_{i=1}^n P_{X_i | X_1 \dots X_{i-1}}(x_i | x_1 \dots x_{i-1}) \\ &= - \sum_{x_1 \dots x_n} \sum_{i=1}^n P_{X_1 \dots X_n}(x_1 \dots x_n) \log P_{X_i | X_1 \dots X_{i-1}}(x_i | x_1 \dots x_{i-1}) = - \sum_{i=1}^n \sum_{x_1 \dots x_n} P_{X_1 \dots X_n}(x_1 \dots x_n) \log P_{X_i | X_1 \dots X_{i-1}}(x_i | x_1 \dots x_{i-1}) \\ &= - \sum_{i=1}^n \sum_{x_1 \dots x_{i-1}} P_{X_1 \dots X_{i-1}}(x_1 \dots x_{i-1}) \log P_{X_i | X_1 \dots X_{i-1}}(x_i | x_1 \dots x_{i-1}) = \sum_{i=1}^n H(X_i | X_1 \dots X_{i-1}) \end{aligned}$$

- For independent $X_1 \dots X_n$, $P_{X_n | X_1 \dots X_{n-1}}(x_n | x_1 \dots x_{n-1}) = P_{X_n}(x_n) \rightarrow H(X_n | X_1 \dots X_{n-1}) = H(X_n)$

$$\therefore H(X_1 \dots X_n) = \sum_{i=1}^n H(X_i)$$

Entropy of functions

- Let X be a RV w/ pmf P_X and $Z = g(X)$ for some function g . Expanding $H(X, Z)$ in two ways,

$$H(X, Z) = H(X) + H(Z | X) = H(Z) + H(X | Z)$$

$$\therefore H(X) = H(Z) + H(X | Z) - H(Z | X)$$

Since $H(Z | X) = H(g(X) | X) = 0$ and $H(X | Z) \geq 0$, we have $H(Z) \geq H(X)$

and the equality holds for functions g w/ one-to-one mapping (so $H(X | Z) = 0$)

* The above holds true for joint pmfs — say we have a one-to-one mapping $\begin{bmatrix} X_1 \\ X_2 \end{bmatrix} \rightarrow \begin{bmatrix} Z_1 \\ Z_2 \end{bmatrix}$,

$$H(X_1, X_2) = H(Z_1, Z_2)$$

Relative entropy

Relative entropy

- The rel. entropy or Kullback-Leibler (KL) divergence between two pmfs P, Q defined on the same alphabet $\mathcal{X}$

$$D(P || Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)}$$

- Rel. entropy $D(P || Q)$ is a measure of distance between distributions P and Q (how similar they are), but is not a true distance (e.g. $D(P || Q) \neq D(Q || P)$)

- Rel. entropy is always non-negative,

$$D(P || Q) \geq 0$$

w/ equality iff $P = Q$.

$$\begin{aligned} \hookrightarrow -D(P || Q) &= \sum_{x \in \mathcal{X}} P(x) \log \frac{Q(x)}{P(x)} = \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \ln \frac{Q(x)}{P(x)} \\ &\leq \frac{1}{\ln 2} \sum_{x \in \mathcal{X}} P(x) \left(\frac{Q(x)}{P(x)} - 1 \right) = \frac{1}{\ln 2} \left(\sum_{x \in \mathcal{X}} Q(x) - \sum_{x \in \mathcal{X}} P(x) \right) = 0 \rightarrow D(P || Q) \geq 0 \end{aligned}$$

For Personal Use Only -bkwk2

Redundancy in source coding

- Often, the true distribution of the source is unknown, and we have to work w/ an estimated source distribution for compression.

code optimal for $\hat{P}$

- Suppose the true pmf of a RV is $P = \{p_1, \dots, p_n\}$ and the estimated pmf is $\hat{P} = \{\hat{p}_1, \dots, \hat{p}_n\}$.
We use $\hat{P}$ to design a compression code whose code lengths l_i satisfy $l_i = \log \frac{1}{\hat{p}_i}$ for $i=1, \dots, n$.

- The average code length L is

$$L = \sum_i p_i l_i = \sum_i p_i \log \frac{1}{\hat{p}_i} = \sum_i p_i \log \frac{p_i}{\hat{p}_i} = \sum_i p_i \log \frac{1}{p_i} + \sum_i p_i \log \frac{p_i}{\hat{p}_i} = H(P) + D(P || \hat{P})$$

→ $D(P || \hat{P})$ is the redundancy for designing code using $\hat{P}$ rather than the true distribution P .

- Suppose that the true distribution P is one from a set of distributions $\mathcal{P}$. We could design a code using a distribution $\hat{P}$ that min. the worst-case redundancy over the set of distributions $\mathcal{P}$.

$$\hat{P}^* = \arg \min_{\hat{P}} \max_{P \in \mathcal{P}} D(P || \hat{P})$$

and the corresponding redundancy $R^* = \max_{P \in \mathcal{P}} D(P || \hat{P}^*)$ is the minimax redundancy.

Hypothesis testing

- Suppose we have data $X_1, \dots, X_n$ and the knowledge that one of the following is true.

$$H_0: X_1, \dots, X_n \sim \text{iid } P.$$

$$H_1: X_1, \dots, X_n \sim \text{iid } Q.$$

Based on the data $X_1, \dots, X_n$, we have to decide whether hypothesis H_0 or H_1 is true.

- A decision rule is a function that maps the data $(X_1, \dots, X_n)$ to a binary decision (0 for H_0 , 1 for H_1)
w/ any decision rule, two kinds of errors can be made

↳ Type I error: H_0 is true but the decision rule chooses H_1 [false alarm]

↳ Type II error: H_1 is true but the decision rule chooses H_0 [missed detection]

- Given the data $X_1, \dots, X_n$, the likelihood ratio (LR) and normalised log-likelihood ratio (LLR) are

$$LR = \frac{Q(X_1, \dots, X_n)}{P(X_1, \dots, X_n)}$$

$$LLR = \frac{1}{n} \log \frac{Q(X_1, \dots, X_n)}{P(X_1, \dots, X_n)}$$

- To test b/wn distributions P and Q , the optimal decision rule is the LR/LLR thresholding rule, i.e.

For some threshold T/t , choose H_1 if $LR \geq T / LLR \geq t$, o/w choose H_0 .

(Note that both tests are equivalent by choosing $T = 2^{nt}$)

- As we increase t , the condition to choose H_1 becomes more stringent → type I error prob. decreases and type II error prob. increases, and vice versa

- As X_i are independent, we can rewrite LLR as $LLR = \frac{1}{n} \sum_{i=1}^n \log \frac{Q(X_i)}{P(X_i)}$.

Applying WLLN on $\log \frac{Q(X_i)}{P(X_i)}$, we have $LLR = \frac{1}{n} \sum_{i=1}^n \log \frac{Q(X_i)}{P(X_i)} \rightarrow E[\log \frac{Q(X)}{P(X)}]$ as $n \rightarrow \infty$.

If H_0 is true, $E[\log \frac{Q(X)}{P(X)}] = \sum_x P(x) \log \frac{Q(x)}{P(x)} = -D(P || Q)$ so $LLR \rightarrow -D(P || Q)$ as $n \rightarrow \infty$.

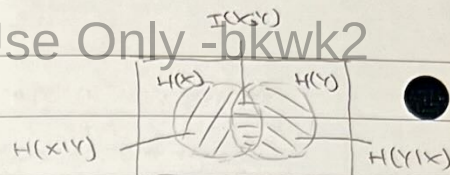
If H_1 is true, $E[\log \frac{Q(X)}{P(X)}] = \sum_x Q(x) \log \frac{Q(x)}{P(x)} = D(Q || P)$ so $LLR \rightarrow D(Q || P)$ as $n \rightarrow \infty$.

(We can rearrange to get the form $Q(X_1, \dots, X_n) \approx P(X_1, \dots, X_n) 2^{-nD(P || Q)}$ [H_0 is true])

$P(X_1, \dots, X_n) \approx Q(X_1, \dots, X_n) 2^{-nD(Q || P)}$ [H_1 is true]

Mutual information

Mutual information



- Consider two RVs X and Y w/ joint pmf $P_{X,Y}$. The mutual information b/w X and Y is

$$I(X;Y) = H(X) - H(X|Y) = H(Y) - H(Y|X)$$

i.e. the reduction in the uncertainty of $X|Y$ when you observe $Y|X$.

- Using the expansion of joint entropy $H(X,Y) = H(X) + H(Y|X) = H(Y) + H(X|Y)$, we can write

$$I(X;Y) = H(X) + H(Y) - H(X,Y) = H(X,Y) - H(X|Y) - H(Y|X)$$

- From above, we can see that mutual information $I(X;Y)$ is symmetric abt X and Y .

$$I(X;Y) = I(Y;X)$$

and also conclude X says as much about Y as Y says about X .

- The mutual information $I(X;Y)$ is also the rel. entropy b/w the joint pmf $P_{X,Y}$ and the product of the marginals $P_X P_Y$, i.e.

$$I(X;Y) = D(P_{X,Y} \| P_X P_Y)$$

$$\begin{aligned} \hookrightarrow I(X;Y) &= H(X) - H(X|Y) = -\sum_x P_X(x) \log P_X(x) + \sum_{x,y} P_{X,Y}(x,y) \log P_{X,Y}(x,y) = \sum_{x,y} P_{X,Y}(x,y) \log \frac{P_{X,Y}(x,y)}{P_X(x)} \\ &= \sum_{x,y} P_{X,Y}(x,y) \log \frac{P_{X,Y}(x,y) P_Y(y)}{P_X(x) P_Y(y)} = \sum_{x,y} P_{X,Y}(x,y) \log \frac{P_{X,Y}(x,y)}{P_X(x) P_Y(y)} = D(P_{X,Y} \| P_X P_Y) \end{aligned}$$

- Mutual information $I(X;Y)$ is non-negative, since $D(P \| Q) \geq 0$ for any pair of pmfs P, Q .

$$I(X;Y) \geq 0$$

$$\therefore I(X;Y) = H(X) - H(X|Y) \geq 0 \rightarrow H(X) \geq H(X|Y)$$

$$= H(Y) - H(Y|X) \geq 0 \rightarrow H(Y) \geq H(Y|X)$$

i.e. conditioning can only reduce the entropy.

- For independent X, Y , $H(X,Y) = H(X) + H(Y) \rightarrow I(X;Y) = H(X) + H(Y) - H(X,Y) = 0$

For $Y=f(X)$, $H(Y|X) = 0 \rightarrow I(X;Y) = H(Y) - H(Y|X) = H(Y)$

Conditional mutual information and chain rule.

- Given X, Y, Z jointly distributed w/ pmf $P_{X,Y,Z}$, the conditional information $I(X;Y|Z)$ is

$$I(X;Y|Z) = H(X|Z) - H(X|Y,Z)$$

$I(X;Y) = H(X) - H(X|Y)$
but everything conditioned w/ Z .

- For RVs X_1, X_2 jointly distributed w/ Y , the chain rule states

$$I(X_1, X_2; Y) = I(X_1; Y) + I(X_2; Y | X_1)$$

In general for any sequence of RVs $X_1, X_2, \dots, X_n$ jointly distributed w/ Y , the chain rule states

$$I(X_1, X_2, \dots, X_n; Y) = \sum_{i=1}^n I(X_i; Y | X_1, \dots, X_{i-1})$$

$$\hookrightarrow I(X_1, X_2, \dots, X_n; Y) = H(X_1, X_2, \dots, X_n) - H(X_1, X_2, \dots, X_n | Y) = \sum_{i=1}^n H(X_i | X_1, \dots, X_{i-1}) - \sum_{j=1}^n H(X_j | Y, X_1, \dots, X_{j-1})$$

$$= \sum_{i=1}^n [H(X_i | X_1, \dots, X_{i-1}) - H(X_i | Y, X_1, \dots, X_{i-1})] = \sum_{i=1}^n I(X_i; Y | X_1, \dots, X_{i-1})$$

where $H(X_1, X_2, \dots, X_n) = \sum_{i=1}^n H(X_i | X_1, \dots, X_{i-1})$ and $H(X_1, X_2, \dots, X_n | Y) = \sum_{j=1}^n H(X_j | Y, X_1, \dots, X_{j-1})$ uses entropy chain rule.

Differential entropy

Differential entropy

- The differential entropy of a continuous RV X w/ pdf f_X is

$$h(X) = \int f_X(u) \log \frac{1}{f_X(u)} du$$

where the integral is evaluated over the support of the RV

- Let $X \sim \text{Unif}[0, a]$. Its differential entropy $h(X)$ is

$$h(X) = \int_0^a f_X(u) \log \frac{1}{f_X(u)} du = \int_0^a \frac{1}{a} \log a \, du = \log a$$

For $a=1$, $h(X)=0$; for $0 < a < 1$, $h(X) < 0$.

- For any RV X , we may interpret its differential entropy $h(X)$ as the uncertainty rel. to that of a $\text{Unif}[0,1]$ RV, which has differential entropy of 0 (so $h(X) < 0$ is possible)

Differential entropy for multiple RVs

- Let X and Y be continuous RVs w/ joint pdf $f_{X,Y}$. The joint differential entropy of X, Y is

$$h(X, Y) = \iint f_{X,Y}(u, v) \log \frac{1}{f_{X,Y}(u, v)} du dv$$

and the conditional differential entropy of X given Y is

$$h(X|Y) = \iint f_{X,Y}(u, v) \log \frac{1}{f_{X,Y}(u|v)} du dv$$

where the integrals are evaluated over the support of the RV.

- Joint differential entropy $h(X, Y)$ obeys a chain rule

$$h(X, Y) = h(X) + h(Y|X) = h(Y) + h(X|Y)$$

- For independent X, Y , $h(Y|X) = h(Y)$ / $h(X|Y) = h(X)$

$$\therefore h(X, Y) = h(X) + h(Y)$$

Relative entropy and mutual information between two densities

- The rel. entropy b/w two densities f and g w/ the same support is

$$D(f||g) = \int f(u) \log \frac{f(u)}{g(u)} du$$

and is always non-negative, $D(f||g) \geq 0$, w/ equality iff $f=g$.

- The mutual information b/w two continuous RVs X, Y is

$$I(X;Y) = h(X) - h(X|Y) = h(Y) - h(Y|X)$$

(equivalently)

$$= h(X) + h(Y) - h(X, Y) = h(X, Y) - h(X|Y) - h(Y|X)$$

and is always non-negative, $I(X;Y) \geq 0$, since $I(X;Y) = D(f_{X,Y} || f_X f_Y)$

* Relative entropy and mutual information have their usual interpretations and properties

+ when computing $h(X)$ and $D(f||g)$ for continuous RV, take advantage of the following pdf properties

$$\int_{\mathcal{X}} f_X(x) dx = 1, \quad \int_{\mathcal{X}} x f_X(x) dx = E[X], \quad \int_{\mathcal{X}} x^2 f_X(x) dx = E[X^2] = \text{Var}(X) + E[X]^2, \quad \int_{\mathcal{X}} (x-\mu)^2 f_X(x) dx = \text{Var}(X)$$

Typicality and AEP

Estimating tail probabilities

- Markov and Chebyshev inequalities are ways to bound tail probabilities w/ limited information about the RV (but these bounds are quite loose)

↳ Markov: non-negative RVs only, requires mean

↳ Chebyshev: general RVs, requires mean and variance.

① Markov's inequality.

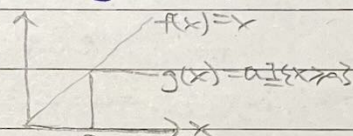
- Markov's inequality states for a non-negative RV X and any $a > 0$,

$$P(X \geq a) \leq \frac{E[X]}{a}$$

(this is only useful for $a > E[X]$, o/w the bound would be greater than 1)

- Define the indicator function $\mathbb{1}_{\{X \geq a\}} = \begin{cases} 1 & \text{if } X \geq a \\ 0 & \text{if } X < a \end{cases}$

Consider the functions $f(x) = x$, $g(x) = a \mathbb{1}_{\{X \geq a\}}$.



For $x \geq 0$, we have $x \geq a \mathbb{1}_{\{X \geq a\}}$, so

$$\sum_{x \geq 0} x P(X=x) \geq \sum_{x \geq 0} a \mathbb{1}_{\{X \geq a\}} P(X=x)$$

$$E[X] \geq a P(X \geq a) \rightarrow P(X \geq a) \leq \frac{E[X]}{a}$$

② Chebyshev's inequality

- Chebyshev's inequality states for any RV X and $a > 0$,

$$P(|X - E[X]| \geq a) \leq \frac{\text{Var}(X)}{a^2}$$

- Consider the RV $Y = |X - E[X]|^2$ and apply Markov's inequality using a^2 .

$$P(Y \geq a^2) \leq \frac{E[Y]}{a^2}$$

$$P(|X - E[X]|^2 \geq a^2) \leq \frac{E[|X - E[X]|^2]}{a^2} \rightarrow P(|X - E[X]| \geq a) \leq \frac{\text{Var}(X)}{a^2}$$

(since $P(|X - E[X]|^2 \geq a^2) = P(|X - E[X]| \geq a)$ and $E[|X - E[X]|^2] = \text{Var}(X)$)

Weak law of large numbers (WLLN)

- Let $X_1, X_2, \dots$ be a sequence of iid RVs w/ finite mean μ , and let $S_n = \frac{1}{n} \sum_{i=1}^n X_i$.

The WLLN states that for any $\epsilon > 0$,

$$\lim_{n \rightarrow \infty} P(|S_n - \mu| \geq \epsilon) = 0$$

or informally, $S_n \rightarrow \mu$ as $n \rightarrow \infty$.

- Assuming the variance σ^2 of each X_i is finite, applying Chebyshev's inequality to S_n gives

$$P(|S_n - \mu| \geq \epsilon) = \frac{\text{Var}(S_n)}{\epsilon^2} = \frac{\sigma^2}{n\epsilon^2}$$

since $\text{Var}(S_n) = \frac{1}{n^2} \text{Var}(\sum_{i=1}^n X_i) = \frac{1}{n^2} \sum_{i=1}^n \text{Var}(X_i) = \frac{1}{n^2} n \sigma^2 = \frac{\sigma^2}{n}$

For any fixed $\epsilon > 0$, $\frac{\sigma^2}{n\epsilon^2} \rightarrow 0$ as $n \rightarrow \infty$, so we get the WLLN.

For Personal Use Only -bkwk2

Typical set and the AEP

- The typical set $A_{\epsilon, n}$ is the set of typical sequences $\{x^n\}$ wrt the pmf P_x is defined as,

$$A_{\epsilon, n} = \{x^n \in \mathcal{X}^n : |-\frac{1}{n} \log P_x(x^n) - H(x)| < \epsilon\}$$

$$\text{where } P_x(x^n) = \prod_{i=1}^n P_x(x_i)$$

- Alternatively, we can rewrite $|-\frac{1}{n} \log P_x(x^n) - H(x)| < \epsilon$ as

$$-\epsilon < -\frac{1}{n} \log P_x(x^n) - H(x) < \epsilon \rightarrow 2^{-n(H(x)+\epsilon)} < P_x(x^n) < 2^{-n(H(x)-\epsilon)}$$

$$\therefore A_{\epsilon, n} = \{x^n \in \mathcal{X}^n : 2^{-n(H(x)+\epsilon)} < P_x(x^n) < 2^{-n(H(x)-\epsilon)}\}$$

this means typical sequences are sequences whose probability is conc. around $2^{-nH(x)}$

- Let $x^n = (x_1, x_2, \dots, x_n)$ be a sequence generated iid $\sim P_x$, i.e. $P(x^n = x^n) = \prod_{i=1}^n P_x(x_i)$,

then for any $\epsilon > 0$,

① $P(x^n \in A_{\epsilon, n}) \rightarrow 1$ as $n \rightarrow \infty$, i.e. any sequence is likely to be typical as $n \rightarrow \infty$.

- Applying the WLLN on $Y_i = -\log P_x(x_i)$ (Note since Y_i are iid, $Y^n = (Y_1, Y_2, \dots, Y_n)$ are iid)

$$\lim_{n \rightarrow \infty} P(|\frac{1}{n} \sum Y_i - E[Y_i]| < \epsilon) = 1$$

- Note $\sum Y_i = -\sum \log P_x(x_i) = -\log \prod P_x(x_i) = -\log P_x(x_1, x_2, \dots, x_n) = -\log P_x(x^n)$

$$\text{and } E[Y_i] = E[-\log P_x(x)] = H(x)$$

Asymptotic equipartition property (AEP)

$$\therefore \lim_{n \rightarrow \infty} P(|-\frac{1}{n} \log P_x(x^n) - H(x)| < \epsilon) = 1 \rightarrow P(x^n \in A_{\epsilon, n}) \rightarrow 1 \text{ as } n \rightarrow \infty$$

② $|A_{\epsilon, n}| \leq 2^{n(H(x)+\epsilon)}$, where $|A_{\epsilon, n}|$ denotes the no. of elements in the typical set $A_{\epsilon, n}$.

- Since the typical set is a subset of all possible sequences,

$$\sum_{x^n \in A_{\epsilon, n}} P_x(x^n) \leq \sum_{x^n \in \mathcal{X}^n} P_x(x^n) = 1$$

From the defn of the typical set $A_{\epsilon, n}$, we have $P_x(x^n) \geq 2^{-n(H(x)+\epsilon)}$

$$1 \geq \sum_{x^n \in A_{\epsilon, n}} P_x(x^n) \geq \sum_{x^n \in A_{\epsilon, n}} 2^{-n(H(x)+\epsilon)} = |A_{\epsilon, n}| 2^{-n(H(x)+\epsilon)}$$

$$\therefore |A_{\epsilon, n}| \leq 2^{n(H(x)+\epsilon)}$$

③ $|A_{\epsilon, n}| \geq (1-\epsilon) 2^{n(H(x)-\epsilon)}$ for sufficiently large n .

- From ①, for sufficiently large n , $P(x^n \in A_{\epsilon, n}) > 1-\epsilon$, so

$$1-\epsilon < P(x^n \in A_{\epsilon, n}) = \sum_{x^n \in A_{\epsilon, n}} P_x(x^n)$$

From the defn of the typical set $A_{\epsilon, n}$, we have $P_x(x^n) \leq 2^{-n(H(x)-\epsilon)}$

$$1-\epsilon < \sum_{x^n \in A_{\epsilon, n}} P_x(x^n) \leq \sum_{x^n \in A_{\epsilon, n}} 2^{-n(H(x)-\epsilon)} = |A_{\epsilon, n}| 2^{-n(H(x)-\epsilon)}$$

$$\therefore |A_{\epsilon, n}| \geq (1-\epsilon) 2^{n(H(x)-\epsilon)}$$

→ In summary, for large n ,

Definition + ①

→ The sequence x^n generated iid $\sim P_x$ will likely be typical, i.e. have probability $\sim 2^{-nH(x)}$

① → The typical set essentially is the set of sequence whose fraction of symbol k is

close to $P(k)$, for all symbols k in the alphabet $\mathcal{X}$.

② + ③

→ The no. of typical sequences is close to $2^{nH(x)}$

For Personal Use Only -bkwk2

Data compression (source coding) and the coding theorem

Data compression (source coding)

- The goal is to compress a source producing symbols $x_1, x_2, \dots \in \mathcal{X}$ that are iid w.p. p_x .

- For any n , a compression code is defined as follows:

↳ To each source sequence $x^n = (x_1, \dots, x_n)$, the code assigns a codeword $c(x^n)$, a unique binary sequence.

↳ Denoting the length of the codeword assigned to x^n as $\ell(x^n)$, the expected code length $E[\ell(x^n)]$ is defined as

$$E[\ell(x^n)] = \sum_{x^n} P(x^n) \ell(x^n)$$

→ We want to find a code (mapping) that has the shortest expected code length possible.

A naive compression code

- Consider the code where we index the $|\mathcal{X}|^n$ possible length n sequences as $\{0, 1, \dots, |\mathcal{X}|^n - 1\}$ using $\lceil \log |\mathcal{X}|^n \rceil$ bits → length of codeword for each sequence is $\lceil \log |\mathcal{X}|^n \rceil = n \lceil \log |\mathcal{X}| \rceil$

- The expected code length is $E[\ell(x^n)] = n \log |\mathcal{X}|$

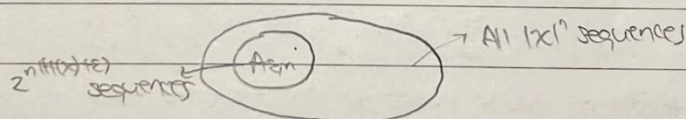
The expected no. of bits per symbol is $\frac{E[\ell(x^n)]}{n} = \log |\mathcal{X}|$

+ This naive compression code acts as a "benchmark" for analysing other codes' performance.

Compression via the typical set.

- Out of all the $|\mathcal{X}|^n$ sequences, we have at most $2^{n(H(X) + \epsilon)}$ typical sequences.

a unique codeword is assigned to each of the $|\mathcal{X}|^n$ sequences



$$x \leq x^n \leq x+1$$

We index each sequence in $A_{\epsilon n}$ using $\lceil \log 2^{n(H(X) + \epsilon)} \rceil$ bits + prefix w/ flag bit 0

→ no. of bits (typical seq.) = $\lceil n(H(X) + \epsilon) \rceil + 1 \leq n(H(X) + \epsilon) + 2$

and index each sequence not in $A_{\epsilon n}$ using $\lceil \log |\mathcal{X}|^n \rceil$ bits + prefix w/ flag bit 1.

→ no. of bits (non-typical seq.) = $\lceil n \log |\mathcal{X}| \rceil + 1 \leq n \log |\mathcal{X}| + 2$

- The expected code length $E[\ell(x^n)]$ is given by

$$\begin{aligned} E[\ell(x^n)] &= \sum_{x^n} P(x^n) \ell(x^n) = \sum_{x^n \in A_{\epsilon n}} P(x^n) \ell(x^n) + \sum_{x^n \notin A_{\epsilon n}} P(x^n) \ell(x^n) \\ &\leq \sum_{x^n \in A_{\epsilon n}} P(x^n) (n(H(X) + \epsilon) + 2) + \sum_{x^n \notin A_{\epsilon n}} P(x^n) (n \log |\mathcal{X}| + 2) \\ &= n(H(X) + \epsilon) \sum_{x^n \in A_{\epsilon n}} P(x^n) + n \log |\mathcal{X}| \sum_{x^n \notin A_{\epsilon n}} P(x^n) + 2 \left(\sum_{x^n \in A_{\epsilon n}} P(x^n) + \sum_{x^n \notin A_{\epsilon n}} P(x^n) \right) \\ &\leq n(H(X) + \epsilon) - 1 + n \log |\mathcal{X}| + 2 + 2 \cdot 1 \\ &= n(H(X) + \epsilon) + n \log |\mathcal{X}| + 2 = n(H(X) + \epsilon') \rightarrow E[\ell(x^n)] \leq n(H(X) + \epsilon') \end{aligned}$$

where $\epsilon' = \epsilon + \epsilon \log |\mathcal{X}| + \frac{2}{n}$ (ϵ' can be made arbitrarily small by choosing small ϵ , large n)

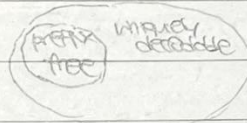
For Personal Use Only -bkwk2

Extension codes, unique decodability and prefix-free codes

- Given a binary code C for alphabet X , the extension code C^n is the code applied symbol-by-symbol to strings in $X^n \subseteq X^*$, i.e. the codeword for x^n is

$$c(x^n) = c(x_1 x_2 \dots x_n) = c(x_1) c(x_2) \dots c(x_n)$$

- The extension code C^n is uniquely decodable if for each binary codeword in C^n , there is only one possible source string that can produce it.
- A code is prefix-free / instantaneously decodable if no codeword is the prefix of another. $\rightarrow$ could be decoded instantaneously symbol-by-symbol.
- Prefix-free codes are uniquely decodable, but not all uniquely decodable codes are prefix-free.



Prefix condition on a tree and Kraft's inequality

- Prefix-free codes have codewords on the leaves of the tree, i.e. doesn't have descendants that are also codewords.
- A full tree of depth $l_{\max}$ has $2^{l_{\max}}$ leaves.
- When a codeword of length $l < l_{\max}$ is created, all its descendants are removed from the tree $\rightarrow$ a set of $2^{l_{\max}-l}$ leaves at depth $l_{\max}$ become usable (the sets of unusable leaves created by the codewords are non-overlapping)
- The total no. of unusable leaves at level $l_{\max}$ for a code w/ word lengths $\{l_1, l_2, \dots, l_n\}$ is $\sum_{i=1}^n 2^{l_{\max}-l_i}$

- Given that the above must be less than the total no. of leaves, we have the inequality

$$\sum_{i=1}^n 2^{l_{\max}-l_i} \leq 2^{l_{\max}} \rightarrow \sum_{i=1}^n 2^{-l_i} \leq 1 \quad [\text{Kraft inequality}]$$

- The Kraft inequality is a necessary and sufficient condition for any prefix-free code w/ lengths $\{l_1, l_2, \dots, l_n\} \Rightarrow$ we can always construct a prefix-free code w/ these lengths.

- In summary, a binary prefix-free code w/ codeword lengths $\{l_1, l_2, \dots, l_n\}$ exists iff

$$\sum_{i=1}^n 2^{-l_i} \leq 1$$

- In fact the Kraft inequality is satisfied by any uniquely-decodable code.

If C^n is any uniquely decodable code which assigns a binary codeword of length $l(x^n)$ to each $x^n \in X^n$, then

$$\sum_{x^n} 2^{-l(x^n)} \leq 1$$

For Personal Use Only -bkwk2

coding theorem for a random variable

- For an extension code, we define the expected codeword length L for each symbol x

$$L = E[l(x)] = \sum_{x \in \mathcal{X}} P_X(x) l(x)$$

- Let X be a RV taking values in $\mathcal{X}$ w/ entropy $H(X)$, the expected codeword length L of any binary uniquely decodable code for X satisfies

$$L \geq H(X)$$

- Denote the probabilities of the symbols as $p_1, p_2, \dots$ and the corresponding codeword lengths as $l_1, l_2, \dots$. Consider $H(X) - L$.

$$H(X) - L = \sum_i p_i \log \frac{1}{p_i} - \sum_i p_i l_i = \sum_i p_i \log \frac{2^{-l_i}}{p_i} = \frac{1}{\ln 2} \sum_i p_i \ln \frac{2^{-l_i}}{p_i}$$

$$\ln x \leq x - 1 \quad \Rightarrow \quad \leq \frac{1}{\ln 2} \sum_i p_i \left(\frac{2^{-l_i}}{p_i} - 1 \right) = \frac{1}{\ln 2} \left(\sum_i 2^{-l_i} - \sum_i p_i \right)$$

$$\text{Kraft inequality} \leq \frac{1}{\ln 2} (1 - 1) = 0 \quad \rightarrow \quad H(X) - L \leq 0$$

coding theorem for blocks of source symbols

- Instead of assigning a codeword to each symbol, suppose the uniquely decodable code assigns binary codewords to blocks of N source symbols.

- Denoting the block as $x^N = (x_1, \dots, x_N)$, we can apply the coding theorem to the code for x^N the expected length of code satisfies

$$E[l(x^N)] \geq H(x^N) \quad \rightarrow \quad \frac{E[l(x^N)]}{N} \geq \frac{H(x^N)}{N}$$

- If X is an iid source, i.e. $x_1, x_2, \dots, x_N$ are iid $\sim P_X$ w/ entropy $H(X)$, then $H(x^N) = NH(X)$, so

$$\frac{E[l(x^N)]}{N} \geq H(X) \quad [1]$$

- Through compression via the typical set, it was shown that we can represent sequences x^N using $NH(X)$ bits on average ($E[l(x^N)] \leq N(H(X) + \epsilon)$)

- Let x^N be iid $\sim P_X$. Fix any $\epsilon > 0$, for sufficiently large N , there exists a code that maps sequences x^N of length n into binary strings such that

$$\frac{E[l(x^N)]}{N} \leq H(X) + \epsilon \quad [2]$$

- [1] and [2] together form Shannon's lossless source coding theorem for iid sources.

↳ You can construct a uniquely decodable code w/ expected length arbitrarily close to the entropy (by taking block length N large enough)

↳ conversely, you cannot construct a uniquely decodable code w/ expected length less than the source entropy

For Personal Use Only -bkwk2

Practical data compression (source coding)

Shannon-Fano coding

- The coding theorem states that the expected code length $L \geq H(X)$. For a source which can take m values w/ probabilities $P_1, \dots, P_m$, we have

$$L = \sum_{i=1}^m P_i l_i \geq \sum_{i=1}^m P_i \log_2 \frac{1}{P_i} = H(X)$$

- For the inequality to be an equality, we need to set $l_i = \log_2 \frac{1}{P_i}$. However, the code word lengths l_i are integers, so we choose

$$l_i = \lceil \log_2 \frac{1}{P_i} \rceil$$

and since $\lceil x \rceil < x+1$, the expected code length is bounded by

$$L = \sum_i P_i l_i < \sum_i P_i (\log_2 \frac{1}{P_i} + 1) = H(X) + 1$$

- checking Kraft's inequality,

$$\sum_i 2^{-l_i} = \sum_i 2^{-\lceil \log_2 \frac{1}{P_i} \rceil} \leq \sum_i 2^{-\log_2 \frac{1}{P_i}} = \sum_i P_i = 1 \leq 1$$

Properties of an optimal prefix-free code

- An optimal prefix-free code has the following properties
 - ↳ The lengths are ordered inversely w/ the probabilities, i.e. if $P_j > P_k$, then $l_j \leq l_k$
 - ↳ The two least probable symbols have the same length and are on neighbouring leaves in the binary tree (i.e. they differ only in the last digit). ↑
not just same level

Huffman coding

- Huffman is an algorithm that gives an optimal prefix-free code for a given set of probabilities. It uses the two properties above and builds the binary tree backwards.

- The algorithm is as follows:

- ↳ 1) Take the two least probable symbols in the alphabet. These two symbols will be given the longest codewords, which equal length and differ only in the last digit.
- ↳ 2) Combine these two symbols into a single symbol and repeat.

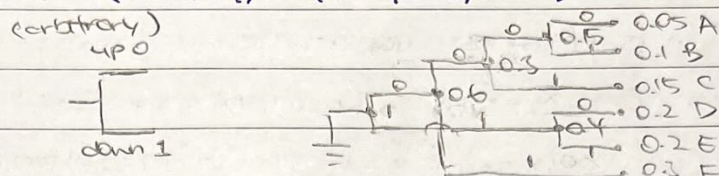
- Huffman coding has the following properties:

- ↳ For a source w/ alphabet size m , the Huffman algorithm req. $m-1$ steps of combining the two smallest probabilities at each stage.
- ↳ Huffman code for a given source may not be unique, but have same expected code length L (can swap 0s and 1s etc.)

- ↳ Huffman code is optimal — given a set of probabilities, there is no prefix-free code that has smaller than expected length than the Huffman code.

For Personal Use Only -bkwk2

- e.g. A source produces symbols from the set $\{A, B, C, D, E, F\}$ with probabilities $\{0.05, 0.1, 0.15, 0.2, 0.2, 0.3\}$, construct a Huffman code.



$F \rightarrow 01, E \rightarrow 11, D \rightarrow 10, C \rightarrow 001, B \rightarrow 0001, A \rightarrow 0000$

Expected code length $L = (0.05 + 0.1) \cdot 4 + 0.15 \cdot 3 + (0.2 + 0.2 + 0.3) \cdot 2 = 2.45$ bits

- * Expected code length L can be found from the sum of "super-symbol" probabilities, i.e. in this example, $L = 1 + 0.6 + 0.3 + 0.15 + 0.4 = 2.45$ bits.

- Huffman codes are optimal for coding a single RV X , and has an expected code length L that satisfies $L < H(X) + 1$ (same as Shannon-Fano)

- However, we typically want to compress a long source sequence $X^N = (X_1 \dots X_N)$.

- Even for an iid source (usually not the case), symbol-by-symbol Huffman coding gives an expected length L closer to $H(X) + 1$ than $H(X)$.

- To reduce the overhead of up to 1 bit/symbol, we could design Huffman codes for k symbols $\rightarrow$ overhead of up to 1 bit / k symbols, i.e. $1/k$ bits / symbol.

- This comes at the expense of increased complexity. For a block of k symbols, the alphabet size is now $|X|^k \rightarrow$ much larger binary tree. Also, we can't decode symbol-by-symbol.
 $\hookrightarrow$ cannot achieve linear complexity $O(n)$ like arithmetic coding.

Interval coding.

- Each symbol can be represented as an interval inside $[0, 1]$, w/ the interval length equal to the symbol probability.
- A source w/ m symbols w/ probabilities $\{p_1, p_2, \dots, p_m\}$ is represented using the m intervals $[0, p_1), [p_1, p_1 + p_2), \dots, [\sum_{i=1}^{m-1} p_i, \sum_{i=1}^m p_i + p_m)$
- The binary codeword for a symbol w/ probability p represented by $[a, a+p)$ can be found by
 - $\hookrightarrow$ 1) Find the largest dyadic interval of the form $[\frac{j}{2^l}, \frac{j+1}{2^l})$ that lies within $[a, a+p)$ ($j, l \in \mathbb{Z}$)
 - $\hookrightarrow$ 2) Take the binary representation of the lower endpoint of the dyadic interval as the codeword (the integer j converted to binary, represented using l bits)
- w/ l bits, the dyadic intervals have length 2^{-l} . As they must lie within the interval of length p ,

$$2^{-l} \leq p \quad \rightarrow \quad l \geq \lceil \log_2 \frac{1}{p} \rceil$$

However, sometimes we need $l = \lceil \log_2 \frac{1}{p} \rceil + 1$ bits for a sufficiently small dyadic interval.

$\rightarrow$ Length of binary codeword for a symbol w/ probability p is either $\lceil \log_2 \frac{1}{p} \rceil$ or $\lceil \log_2 \frac{1}{p} \rceil + 1$ bits.

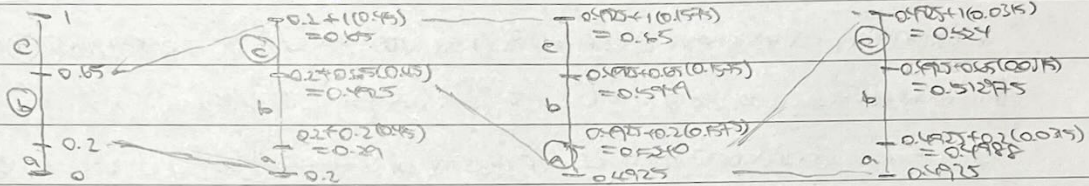
- The expected code length L is bounded as $L = \sum p_i l_i \leq \sum p_i (\lceil \log_2 \frac{1}{p_i} \rceil + 1) < H(X) + 2$

- If the binary codeword c was a prefix of another codeword d , then the interval $[0.d0, 0.d1)$ would be a sub-interval $[0.c0, 0.c1)$ $\rightarrow$ interval coding is prefix-free

Arithmetic coding

- consider a source producing symbols $X^N = (x_1, x_2, \dots, x_N)$ iid $\sim P_X$. Each length N string $X^N = (x_1, x_2, \dots, x_N)$ is represented a disjoint interval w length equal to the probability of the string.
- the interval for $(x_1=x_1, x_2=x_2)$ is a sub-interval of the interval for $x_1=x_1$ $\forall$ symbols x_i . The interval for $(x_1=x_1, x_2=x_2, x_3=x_3)$ is a subinterval of the interval for $(x_1=x_1, x_2=x_2)$ $\forall$ symbols, etc.
- e.g: A source produces symbols from the set $\{a, b, c\}$ w/ probabilities $\{0.2, 0.45, 0.35\}$.

Perform arithmetic coding to find the codeword for the sequence $(x_1=b, x_2=c, x_3=a, x_4=c)$



→ the interval representing (b, c, a, c) is $[0.9925, 0.99775]$

$$(\text{interval width} = 0.99775 - 0.9925 = 0.00525 = 0.45 \times 0.35 \times 0.2 \times 0.35)$$

- as w/ interval coding, the binary codeword is obtained by finding the (unique) dyadic interval of the form $[\frac{j}{2^L}, \frac{j+1}{2^L})$ that is contained within the interval.
- the codeword length L is either $\lceil \log_2 \frac{1}{P_X(X^N)} \rceil$ or $\lceil \log_2 \frac{1}{P_X(X^N)} \rceil + 1$
- To decode, use the binary codeword to sequentially zoom in to the interval, and decode symbols as you go along.
- the expected code length $E[L(X^N)]$ is bounded as $E[L(X^N)] = \sum_x P_X(x^N) L(x^N) < \sum_x P_X(x^N) (\lceil \log_2 \frac{1}{P_X(x^N)} \rceil + 1) < H(X^N) + 2$
therefore the expected code length per symbol is $\frac{E[L(X^N)]}{N} < \frac{H(X^N)}{N} + \frac{2}{N}$
For $x_1, x_2, \dots, x_N$ iid $\sim P_X$, $H(X^N) = NH(X)$, so $\frac{E[L(X^N)]}{N} < H(X) + \frac{2}{N}$
- w/ growing N , arithmetic coding can achieve expected code length per symbol $\frac{E[L(X^N)]}{N}$ arbitrarily close to the source entropy.

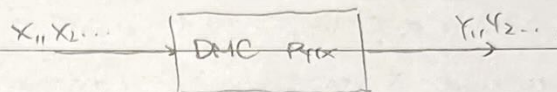
- for non-iid sources where the probability distribution of a symbol x_N depends on $x_1, \dots, x_{N-1}$, arithmetic coding can be extended by using the conditional distribution $x_N | x_1=x_1, \dots, x_{N-1}=x_{N-1}$.

Summary of practical data compression.

- For 1 symbol, $L < L_H \leq L_{SF} < H(X) + 1$, $L < L_I < H(X) + 2$
For blocks of N symbols, each iid $\sim P_X$, $\frac{E[L(X^N)]}{N} < \frac{E[L(X^N)]_H}{N} < \frac{E[L(X^N)]_{SF}}{N} < H(X) + \frac{1}{N}$, $\frac{E[L(X^N)]}{N} < \frac{E[L(X^N)]_A}{N} < H(X) + \frac{2}{N}$
- Shannon-Fano is mainly for academic interest - its compression performance is often significantly worse than other algorithms.
- Huffman is optimal for 1 symbol and has good error resilience.
- Arithmetic has the best compression rate for strings of symbols but suffers from catastrophic error propagation. Its complexity scales linearly w/ sequence length

Discrete and continuous channels

Discrete memoryless channel (DMC)



- A DMC is a system consisting of an i/p alphabet $\mathcal{X}$, o/p alphabet $\mathcal{Y}$, and a set of transition probabilities

$$P_{Y|X}(b|a) = P(Y=b | X=a) \quad \forall a \in \mathcal{X} \text{ and } b \in \mathcal{Y}$$

- The DMC is "memoryless" as the channel acts independently on each i/p. For each time instant k ,

$$P(Y_k=y | X_k=x, X_{k-1} \dots X_1, Y_{k-1} \dots Y_1) = P_{Y|X}(y|x)$$

i.e. Given all the past, the current o/p only depends on the current i/p.

channel capacity

- The channel capacity of a DMC is defined as

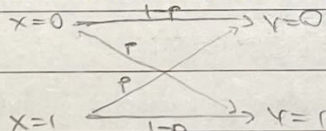
$$C = \max_{P_X} I(X;Y)$$

where $P_{Y|X}$ is defined by the DMC.

- Each i/p distribution P_X yields a diff. joint distribution on X, Y , given by $P_X P_{Y|X}$.

Binary symmetric channel (BSC)

- The BSC w/ crossover probability p is as follows



$P_{Y X}$	Y	
	0	1
X	0	$1-p$ p
	1	p $1-p$

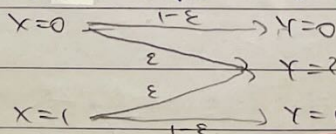
- The capacity of BSC(p) is $C = 1 - H_2(p)$

$$\begin{aligned} I(X;Y) &= H(Y) - H(Y|X) = H(Y) - \sum_{x \in \{0,1\}} P_X(x) H(Y|X=x) \\ &= H(Y) - H_2(p) \leq \log_2 2 - H_2(p) = 1 - H_2(p) \rightarrow I(X;Y) \leq 1 - H_2(p) \end{aligned}$$

consider $P_X = (\frac{1}{2}, \frac{1}{2}) \rightarrow P_Y = (\frac{1}{2}(1-p) + \frac{1}{2}p, \frac{1}{2}p + \frac{1}{2}(1-p)) = (\frac{1}{2}, \frac{1}{2}) \rightarrow H(Y) = 1$ (max. value obtained)

Binary erasure channel (BEC) — model packet loss in networks

- The BEC w/ erasure probability ϵ is as follows



Prix	Y		
	0	ϵ	1
0	$1-\epsilon$	ϵ	0
X 1	0	ϵ	$1-\epsilon$

- The capacity of BEC(ϵ) is $C = 1 - \epsilon$

$$I(X;Y) = H(Y) - H(Y|X) = H(Y) - \sum_{x \in \{0,1\}} P_X(x) H(Y|X=x) = H(Y) - \sum_{x \in \{0,1\}} P_X(x) H_2(\epsilon) = H(Y) - H_2(\epsilon)$$

consider $P_X = (\alpha, 1-\alpha) \rightarrow P_Y = (\alpha(1-\epsilon), \alpha\epsilon + (1-\alpha)\epsilon, (1-\alpha)(1-\epsilon)) = (\alpha(1-\epsilon), \epsilon, (1-\alpha)(1-\epsilon))$

$$\rightarrow H(Y) = -\alpha(1-\epsilon) \log_2(1-\epsilon) - \epsilon \log_2 \epsilon - (1-\alpha)(1-\epsilon) \log_2((1-\alpha)(1-\epsilon)) = H_2(\epsilon) + (1-\epsilon) H_2(\alpha)$$

max. when $\alpha = \frac{1}{2} \rightarrow H(Y) = H_2(\epsilon) + 1 - \epsilon \rightarrow C = 1 - \epsilon$

For Personal Use Only -bkwk2

Additive White Gaussian Noise (AWGN) channel.

- The continuous-time AWGN channel is $Y(t) = X(t) + N(t)$. We assume the channel

→ 1) Input $X(t)$ is power-limited to P (so average energy over time $T \leq P$ for large T), as battery power is limited + limit interference w/ other users.

→ 2) Input $X(t)$ is bandlimited to W (FT of $X(t)$, $X(\omega)$ is zero outside $[-W, W]$) as BW is a scarce resource

→ 3) Noise $N(t)$ is white gaussian, as it is a good approx to sum of many zero-mean RV (CLT), has the highest entropy for a given variance, and is easy to analyse.

- Through the Shannon-Nyquist sampling theorem, it can be shown that the continuous-time channel $Y(t) = X(t) + N(t)$ w/ power/BW constraint P/W is equivalent to the discrete-time channel

$$Y_k = X_k + Z_k \quad k=1, 2, \dots$$

where $\sum_{k=1}^N X_k^2 \leq P$, $Z_k \text{ iid } \sim N(0, \sigma^2)$ and we sampled at a freq. $\geq 2W$.

- Let $X \sim N(\mu, \sigma^2)$. The pdf $\phi(x)$ is $\phi(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$ and thus the differential entropy $h(X)$ is

$$h(X) = - \int \phi(x) \log \phi(x) dx = -\frac{1}{\ln 2} \int \phi(x) \left[-\frac{(x-\mu)^2}{2\sigma^2} - \frac{1}{2} \ln 2\pi\sigma^2 \right] dx = \frac{1}{\ln 2} \left[\frac{\int (x-\mu)^2 \phi(x) dx}{2\sigma^2} + \frac{1}{2} \ln 2\pi\sigma^2 \right]$$

$$= \frac{1}{2\ln 2} [\ln e + \ln 2\pi\sigma^2] = \frac{1}{2} \log 2\pi e \sigma^2 \rightarrow h(X) = \frac{1}{2} \log 2\pi e \sigma^2$$

- The capacity of discrete-time AWGN channel is $C = \frac{1}{2} \log(1 + \frac{P}{\sigma^2})$ [only depends on SNR]

$$\rightarrow I(X; Y) = h(Y) - h(Y|X) = h(Y) - h(X + Z|X) = h(Y) - h(Z|X) = h(Y) - h(Z)$$

$$\text{Note } E[Y^2] = E[(X+Z)^2] = E[X^2] + 2E[XZ] + E[Z^2] = E[X^2] + 2E[X]E[Z] + \sigma^2$$

$$\text{Power restriction: } E[X^2] \leq P \rightarrow E[Y^2] \leq P + \sigma^2.$$

$h(Z|X) = h(Z)$ since X, Z are independent

Among all RVs Y w/ $E[Y^2] \leq P + \sigma^2$, max. differential entropy is achieved when $Y \sim N(0, P + \sigma^2)$ [G]

Therefore we have $h(Y) \leq \frac{1}{2} \log 2\pi e (P + \sigma^2)$, w/ equality if $Y \sim N(0, P + \sigma^2)$, i.e. $X \sim N(0, P)$

Also, as shown above, $h(Z) = \frac{1}{2} \log 2\pi e \sigma^2$, so

since for $X \sim N(0, P)$, $Z \sim N(0, \sigma^2)$ and $Y = X + Z \rightarrow Y \sim N(0, P + \sigma^2)$

$$I(X; Y) = h(Y) - h(Z) = \frac{1}{2} \log 2\pi e (P + \sigma^2) - \frac{1}{2} \log 2\pi e \sigma^2 = \frac{1}{2} \log(1 + \frac{P}{\sigma^2})$$

→ To show [G], consider $D(f_X || \phi)$, where f_X is the pdf of RV X , power restricted to $\sigma^2 E[X^2] \leq P$, ϕ is the pdf of $N(0, \sigma^2)$, i.e. $\phi(x) = \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}}$

$$D(f_X || \phi) = \int_{-\infty}^{\infty} f_X(x) \log \frac{f_X(x)}{\phi(x)} dx = \int_{-\infty}^{\infty} f_X(x) \log f_X(x) dx - \int_{-\infty}^{\infty} f_X(x) \log \frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} dx$$

$$= -h(X) + \frac{1}{2} \log 2\pi \sigma^2 \int_{-\infty}^{\infty} f_X(x) dx + \frac{1}{2\sigma^2} \log e \int_{-\infty}^{\infty} x^2 f_X(x) dx = -h(X) + \frac{1}{2} \log 2\pi \sigma^2 + \frac{\log e}{2\sigma^2} E[X^2]$$

$$D(f_X || \phi) \geq 0, \text{ so } -h(X) + \frac{1}{2} \log 2\pi \sigma^2 + \frac{\log e}{2\sigma^2} E[X^2] \geq 0$$

$$\therefore h(X) \leq \frac{1}{2} \log 2\pi \sigma^2 + \frac{\log e}{2\sigma^2} E[X^2] \leq \frac{1}{2} \log 2\pi \sigma^2 + \frac{\log e}{2\sigma^2} P = \frac{1}{2} \log 2\pi e \sigma^2$$

where we have equality when $E[X^2] = \sigma^2$, i.e. $X \sim N(0, \sigma^2)$.

- If the channel has BW W , it can be used $2W$ times per second $\rightarrow$ capacity in bits/s

(rather than bits/transmission) is $C = 2W \cdot \frac{1}{2} \log(1 + \frac{P}{\sigma^2}) = W \log(1 + \frac{P}{\sigma^2})$.

For Personal Use Only -bkwk2

Strategy for finding capacity for dtr DMC.

- Channel capacity C is defined as $C = \max_{P_X} I(X;Y)$ for a DMC $P_{Y|X}$. It is found as follows:

↳ 1) $I(X;Y) = H(Y) - H(Y|X)$ [$I(X;Y) = H(X) - H(X|Y)$ only useful for noiseless binary channel]

↳ 2) Simplify $H(Y|X)$ if possible, o/w use $H(Y|X) = \sum_{x \in \mathcal{X}} P_X(x) H(Y|X=x)$.

* For symmetric channels (rows of prob. transition matrix are permutations of each other, equal col. sums),

then $H(Y|X=x) = H(\zeta) \forall x$, where $H(\zeta)$ is the entropy of a row in the prob. transition matrix

↳ optimal P_X may not be unique!

↳ 3) Consider $P_X = (\alpha, 1-\alpha)$ or similar to find P_Y , then find $H(Y)$ in terms of α .

Find max. of $I(X;Y)$ by differentiating wrt α and set to 0, i.e. $\alpha^* = \arg \max_{\alpha} I(X;Y|\alpha)$

- In general, "massage" the channel into a familiar dtr (e.g. BSC, BEC, AGWNN)

effective P ↑ effective SUR (E[$\log P_X$]/E[$\log P_Y$])

Joint typicality and AEP

Joint typical set and the AEP.

- The joint typical set $A_{\epsilon,n}$ is the set of jointly typical sequences $\{(x^n, y^n)\}$ wrt the joint pmf $P_{X,Y}$ is defined as

$$A_{\epsilon,n} = \{(x^n, y^n) \in \mathcal{X} \times \mathcal{Y} : |-\frac{1}{n} \log P_X(x^n) - H(X)| < \epsilon, \\ |-\frac{1}{n} \log P_Y(y^n) - H(Y)| < \epsilon, \\ |-\frac{1}{n} \log P_{X,Y}(x^n, y^n) - H(X,Y)| < \epsilon\}$$

where $P_X(x^n) = \prod_{i=1}^n P_X(x_i)$, $P_Y(y^n) = \prod_{i=1}^n P_Y(y_i)$, $P_{X,Y}(x^n, y^n) = \prod_{i=1}^n P_{X,Y}(x_i, y_i)$

- Alternatively, we can rewrite the joint typical set $A_{\epsilon,n}$ as

$$A_{\epsilon,n} = \{(x^n, y^n) \in \mathcal{X} \times \mathcal{Y} : 2^{-n(H(X)+\epsilon)} < P_X(x^n) < 2^{-n(H(X)-\epsilon)}, \\ 2^{-n(H(Y)+\epsilon)} < P_Y(y^n) < 2^{-n(H(Y)-\epsilon)}, \\ 2^{-n(H(X,Y)+\epsilon)} < P_{X,Y}(x^n, y^n) < 2^{-n(H(X,Y)-\epsilon)}\}$$

i.e. jointly typical sequences have probabilities $P_X(x^n) \sim 2^{-nH(X)}$, $P_Y(y^n) \sim 2^{-nH(Y)}$, $P_{X,Y}(x^n, y^n) \sim 2^{-nH(X,Y)}$.

- Let (X^n, Y^n) be a pair of sequences generated i.i.d. $\sim P_{X,Y}$, i.e. $P(X^n = x^n, Y^n = y^n) = \prod_{i=1}^n P_{X,Y}(x_i, y_i)$, then for any $\epsilon > 0$,

① $P((X^n, Y^n) \in A_{\epsilon,n}) \rightarrow 1$ as $n \rightarrow \infty$

- Proved by showing $P(|-\frac{1}{n} \log P_X(x^n) - H(X)| < \epsilon) \rightarrow 1$ as $n \rightarrow \infty$ — joint AEP

$$P(|-\frac{1}{n} \log P_Y(y^n) - H(Y)| < \epsilon) \rightarrow 1 \text{ as } n \rightarrow \infty$$

$$P(|-\frac{1}{n} \log P_{X,Y}(x^n, y^n) - H(X,Y)| < \epsilon) \rightarrow 1 \text{ as } n \rightarrow \infty. \text{ using WLLN as for 1 variable.}$$

② $|A_{\epsilon,n}| \leq 2^{n(H(X,Y)+\epsilon)}$

- Proved by considering $A_{\epsilon,n} \subseteq \mathcal{U}$ and $P_{X,Y}(x^n, y^n) \geq 2^{-n(H(X,Y)+\epsilon)}$ as for 1 variable

③ If $(\tilde{X}^n, \tilde{Y}^n)$ are a pair of sequences generated i.i.d. $\sim P_X P_Y$, then $P((\tilde{X}^n, \tilde{Y}^n) \in A_{\epsilon,n}) \leq 2^{-n(I(X;Y)-3\epsilon)}$

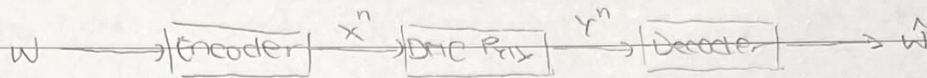
$$P((\tilde{X}^n, \tilde{Y}^n) \in A_{\epsilon,n}) = \frac{|A_{\epsilon,n}(P_X P_Y)|}{|A_{\epsilon,n}(P_X)| |A_{\epsilon,n}(P_Y)|} \approx \frac{2^{nH(X,Y)}}{2^{nH(X)} 2^{nH(Y)}} = 2^{-n(I(X;Y)-3\epsilon)}$$

$$\text{Upper bound, } P((\tilde{X}^n, \tilde{Y}^n) \in A_{\epsilon,n}) = \frac{|A_{\epsilon,n}(P_X P_Y)|}{|A_{\epsilon,n}(P_X)| |A_{\epsilon,n}(P_Y)|} \leq \frac{2^{nH(X,Y)+\epsilon}}{2^{nH(X)-\epsilon} 2^{nH(Y)-\epsilon}} = 2^{-n(I(X;Y)-3\epsilon)}$$

* consider a grid, as in lecture 7

For Personal Use Only -bkwk2

channel code



- We use the channel n times to transmit k information bits. We can think of each sequence of k -bits as indexing a message w in the set $\{1, \dots, 2^k\}$. $\rightarrow k \text{ bits} \leftrightarrow 2^k \text{ messages}$

- The rate R of the code is $R = \frac{k}{n}$ bits / transmission.

- A (n, k) channel code of rate R for the channel $(X, Y, P_{Y|X})$ consists of

$\hookrightarrow$ A set of messages $\{1, \dots, 2^k\}$.

$\hookrightarrow$ An encoding function $x^n: \{1, \dots, 2^k\} \rightarrow X^n$ that assigns a codeword to each message. The set of codewords $\{x^n(1), x^n(2), \dots, x^n(2^k)\}$ is the codebook.

$\hookrightarrow$ A decoding function $g: Y^n \rightarrow \{1, \dots, 2^k\}$ which produces a guess of the transmitted message for each received vector.

- The maximal probability of error of the code is defined as

$$P_{\max} = \max_{j \in \{1, \dots, 2^k\}} P(\hat{w} \neq j | w = j)$$

and the average probability of error of the code is

$$P_e = \frac{1}{2^k} \sum_{j=1}^{2^k} P(\hat{w} \neq j | w = j)$$

where w and $\hat{w}$ denote the transmitted and decoded messages respectively.

Compression via the joint typical set.

- For the general DMC, we fix on i.i.p pmf P_X . Together w/ the channel $P_{Y|X}$, we can find $P_{X,Y}, P_Y$.

$$P_{X,Y} = P_X P_{Y|X}$$

$$P_Y = \sum_x P_X P_{Y|X}$$

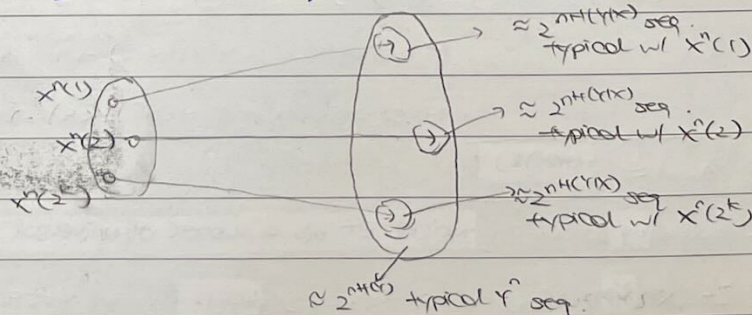
- If our 2^k n -length codewords $x^n(1) \dots x^n(2^k)$ are generated i.i.d $\sim P_X$, then when $x^n(j)$ is transmitted, the set of highly likely Y^n has approx. $2^{nH(Y|X)}$ sequences for each $j \in \{1, \dots, 2^k\}$.

- There are approx. $2^{nH(Y)}$ typical Y^n sequences, so this means we can get $\frac{2^{nH(Y)}}{2^{nH(Y|X)}} = 2^{nI(X;Y)}$ non-intersecting sets $\rightarrow$ each mapped from a $j \in \{1, \dots, 2^k\}$.

$$\therefore 2^k = 2^{nR} = 2^{nI(X;Y)}$$

$\rightarrow$

$$R = I(X;Y)$$

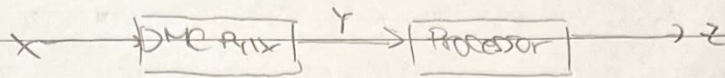


$$2^k \text{ non-intersecting i/p sets} \leftrightarrow \frac{2^{nH(Y)}}{2^{nH(Y|X)}} = 2^{nI(X;Y)} \text{ non-intersecting o/p sets.}$$

For Personal Use Only -bkwk2

channel coding theorem

Data processing inequality



- R.V.s X, Y, Z form a Markov chain if their joint pmf can be written as.

$$P_{X,Y,Z} = P_X P_{Y|X} P_{Z|Y} \quad \leftarrow \text{rather than } P_{X,Y,Z} = P_X P_{Y|X} P_{Z|X,Y}$$

i.e. the conditional distribution of Z given X, Y depends only on Y , $P_{Z|X,Y} = P_{Z|Y}$.

- The data processing inequality states if $X \rightarrow Y \rightarrow Z$ form a Markov chain, then

$$I(X;Y) \geq I(X;Z)$$

i.e. processing the data Y cannot increase the information about X .

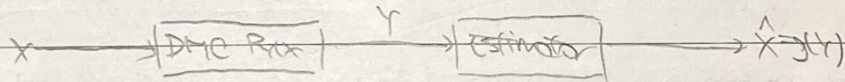
↳ Expanding $I(X;Y,Z)$ in two different ways,

$$I(X;Y,Z) = I(X;Y) + I(X;Z|Y) = I(X;Z) + I(X;Y|Z)$$

$$\text{Note } I(X;Z|Y) = H(Z|Y) - H(Z|X,Y) = H(Z|Y) - H(Z|Y) = 0$$

$$\text{since } I(X;Y|Z) \geq 0, \quad I(X;Y) = I(X;Z) + I(X;Y|Z) \rightarrow I(X;Y) \geq I(X;Z)$$

Fano's inequality



- We want to estimate X by observing a correlated RV Y .

- The probability of error of an estimator $\hat{X} = g(Y)$ is $P_e = P(\hat{X} \neq X)$

- Given X takes values from $\mathcal{X}$, Fano's inequality states for any estimator $\hat{X}$ s.t. $X \rightarrow Y \rightarrow \hat{X}$ form a Markov chain, the probability of error $P_e = P(\hat{X} \neq X)$ satisfies

$$1 + P_e \log |\mathcal{X}| \geq H(X|\hat{X}) \geq H(X|Y) \quad \text{or} \quad P_e \geq \frac{H(X|Y) - 1}{\log |\mathcal{X}|}$$

↳ define an error variable $E = \begin{cases} 1 & \text{if } \hat{X} \neq X \\ 0 & \text{if } \hat{X} = X \end{cases}$ w/ probability P_e w/ probability $1 - P_e$.

$$H(E, X|\hat{X}) = H(X|\hat{X}) + H(E|X, \hat{X}) = H(E|\hat{X}) + H(X|\hat{X}, E)$$

Note $H(E|X, \hat{X}) = 0$ as E is a function of X and $\hat{X}$

$$H(E|\hat{X}) \leq H(E) = H_2(P_e) \leq 1 \quad \text{as conditioning can only reduce entropy, } H_2(p) \leq 1.$$

$$H(X|\hat{X}, E) = \sum_{e \in \{0,1\}} P_e(e) H(X|\hat{X}, E=e) = P_e(0) H(X|\hat{X}, E=0) + P_e(1) H(X|\hat{X}, E=1)$$

$\leftarrow$ actually $\log 2 = 1$ to be precise.

$$= (1 - P_e) 0 + P_e H(X|\hat{X}, E=1) \leq P_e \log |\mathcal{X}| \rightarrow H(X|\hat{X}, E) \leq P_e \log |\mathcal{X}|$$

$$\therefore H(X|\hat{X}) = H(X|\hat{X}) + H(X|\hat{X}, E) \leq 1 + P_e \log |\mathcal{X}| \quad [\text{LHS of Fano's inequality}].$$

↳ Data processing inequality states that $I(X; \hat{X}) \leq I(X; Y)$,

$$H(X) - H(X|\hat{X}) \leq H(X) - H(X|Y)$$

$$H(X|Y) \leq H(X|\hat{X}) \quad [\text{RHS of Fano's inequality}].$$

For Personal Use Only -bkwk2

channel coding theorem.

C is a sharp threshold.

- The channel coding thm. states for a DMC w/ capacity C , all rates less than C are achievable

↳ 1) Fix $R < C$ and pick any $\epsilon > 0$. Then for sufficiently large n , there exists a length- n code of rate R w/ maximal probability of error less than ϵ .

↳ 2) Conversely, any sequence of length- n codes of rate R w/ average/maximal probability of error $P_e/P_{e,\max} \rightarrow 0$ as $n \rightarrow \infty$ must have $R \leq C$.

Proof of first half of channel coding theorem.

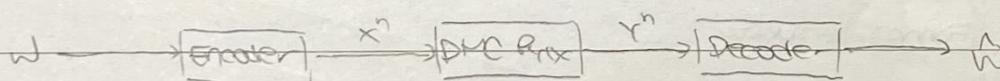
- Fix rate $R < C$ and input pmf P_X . We generate each of the 2^{nR} codewords independently according to the distribution $P(X^n(j)) = \prod_{i=1}^n P_X(x_i)$ for $j=1 \dots 2^{nR}$.

- We can think of the codebook $\mathcal{B}$ as a $2^{nR} \times n$ matrix.

$$\mathcal{B} = \begin{bmatrix} x_1(1) & x_2(1) & \dots & x_n(1) \\ x_1(2) & x_2(2) & \dots & x_n(2) \\ \vdots & \vdots & \ddots & \vdots \\ x_1(2^{nR}) & x_2(2^{nR}) & \dots & x_n(2^{nR}) \end{bmatrix} \begin{matrix} \leftarrow X^n(1) \\ \leftarrow X^n(2) \\ \\ \leftarrow X^n(2^{nR}) \end{matrix}$$

where each entry of the matrix is chosen iid $\sim P_X$.

- The probability that we generate a particular codebook $\{x^n(1) \dots x^n(2^{nR})\}$ is $P(\mathcal{B}) = \prod_{w=1}^{2^{nR}} \prod_{i=1}^n P(x_i(w))$



- The codebook $\mathcal{B}$ is revealed to both sender and receiver, who also know the channel transition matrix $P_{Y|X}$. To transmit message w , the encoder sends $x^n(w)$ over the channel.

- The receiver receives a sequence y^n generated according to $P(y^n | x^n(w)) = \prod_{i=1}^n P(y_i | x_i(w))$

From y^n , the receiver guesses which message was sent, $\hat{w}$.
 (for uniform prior on message) (X and Y maximising this (max-likelihood decoder) is the optimal decoder)

- Using joint typical decoding, the decoder declares that the message $\hat{w}$ was sent if both
(i) $(x^n(\hat{w}), y^n)$ is joint typical wrt P_{XY} , (ii) there is no other message $\hat{w}' \neq \hat{w}$ s.t. $(x^n(\hat{w}'), y^n)$ is joint-typical wrt P_{XY} . If no such $\hat{w}$ is found, an error is declared.

- The average probability of error for a given codebook $\mathcal{B}$ is $P_e = \frac{1}{2^{nR}} \sum_{j=1}^{2^{nR}} P(\hat{w} \neq j | w=j)$

$$\begin{aligned} \text{Averaging this over all the codebooks, } \bar{P}_e &= \frac{1}{2^{nR}} \sum_{\mathcal{B}} \sum_{j=1}^{2^{nR}} P(\hat{w} \neq j | w=j, \mathcal{B}) P(\mathcal{B}) \\ &= \frac{1}{2^{nR}} \sum_{j=1}^{2^{nR}} \sum_{\mathcal{B}} P(\hat{w} \neq j | w=j, \mathcal{B}) P(\mathcal{B}) \end{aligned}$$

The inner sum is the same for all codewords w because each element in the codebook $\mathcal{B}$ is generated iid $\sim P_X$, so w/o loss of generality, $\bar{P}_e = \sum_{\mathcal{B}} P(\hat{w} \neq 1 | w=1, \mathcal{B}) P(\mathcal{B})$

- Assuming $w=1$ was transmitted, there are two sources of error: (i) $(x^n(1), y^n)$ is joint typical wrt P_{XY} . (ii) $(x^n(\hat{w}), y^n)$ is joint typical wrt P_{XY} for some $\hat{w} \neq 1$.

- Let E_j be the event that $(x^n(j), y^n)$ is joint typical wrt P_{XY} .

$$\bar{P}_e = P(\bar{E}_1 \cup (\bigcup_{j=2}^{2^{nR}} E_j)) \leq P(\bar{E}_1) + \sum_{j=2}^{2^{nR}} P(E_j)$$

For Personal Use Only -bkwk2

- It can be shown that $P(\bar{E}_1)$ is small.

↳ $X^n(1)$ is iid $\sim P_X$ and the channel generates Y^n (symbol by symbol) according to $P_{Y|X}(y_i|x_i(1))$

for $i=1..n \rightarrow (X^n(1), Y^n)$ is generated iid $\sim P_X P_{Y|X} = P_{X,Y}$.

⇒ Joint AEP implies that $P(\bar{E}_1) \leq \epsilon$ for sufficiently large n .

- It can be shown that $\sum_{i=2}^{2^{nR}} P(\bar{E}_i)$ is small.

↳ For $j \neq 1$, $X^n(j)$ is generated independently from $X^n(1)$, and Y^n is obtained by passing $X^n(1)$ through the channel $\rightarrow X^n(j)$ and Y^n are independent.

↳ $X^n(j)$ is iid $\sim P_X$ and Y^n is iid $\sim P_Y$. $\rightarrow$ Joint AEP states the probability $X^n(j)$ and Y^n are

jointly typical wrt $P_{X,Y}$ is $\leq 2^{-n(I(X;Y) - 3\epsilon)}$ (for each $j \neq 1$)

⇒ $\sum_{i=1}^{2^{nR}} P(\bar{E}_i) \leq (2^{nR} - 1) \cdot 2^{-n(I(X;Y) - 3\epsilon)} \leq 2^{nR} \cdot 2^{-n(I(X;Y) - 3\epsilon)} = \epsilon$, provided $R < I(X;Y) - 3\epsilon$.

- combining the above, we get $\bar{P}_e \leq P(\bar{E}_1) + \sum_{i=2}^{2^{nR}} P(\bar{E}_i) \leq 2\epsilon$.

- We choose P_X s.t. $I(X;Y)$ is maximised $\rightarrow I(X;Y)^* = C$.

- As we have shown $\bar{P}_e = \frac{1}{2^{nR}} \sum_{j=1}^{2^{nR}} \sum_{\hat{w} \neq j} P(\hat{w}=j | W=j) P(B) \leq 2\epsilon$, there exists a codebook B^* where $P_e(B^*) = \frac{1}{2^{nR}} \sum_{j=1}^{2^{nR}} P(\hat{w} \neq j | W=j, B=B^*) \leq 2\epsilon$.

- For B^* , the average probability of error $P_e(B^*) \leq 2\epsilon$, therefore at least half of the messages have probability of error $\leq \epsilon$. Denote the new codebook w/ only the good half of B^* using B^{**} .

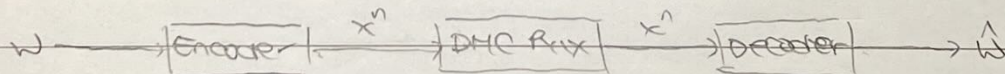
- This codebook B^{**} has a maximal probability of error $P_{e,\max} \leq \epsilon$, and $2^{nR}/2$ codewords.

It has a rate $= \frac{\log 2^{nR/2}}{n} = R - \frac{1}{n} < I(X;Y)^* - 3\epsilon - \frac{1}{n} = C - 3\epsilon - \frac{1}{n}$, say rate $= C - 4\epsilon - \frac{1}{n}$.

⇒ First half of the channel coding thm.

Proof of the second half of channel coding theorem.

- Consider a length n , rate R channel code (i.e. 2^{nR} codewords).



$\hat{w}$ is a guess of w based on Y^n , and we will assume the prior distribution $w \sim \text{Unif}[1, 2^{nR}]$.

- The probability of error of an estimator $\hat{w}$ is $P_e = P(\hat{w} \neq w) = \frac{1}{2^{nR}} \sum_{j=1}^{2^{nR}} P(\hat{w} \neq j | w=j)$

Applying Fano's inequality, we get $H(\hat{w}|w) \leq 1 + P_e \log 2^{nR} = 1 + P_e nR$.

- It can be shown that for a DMC w/ capacity C , $I(X;Y) \leq nC$ regardless the distribution of X^n .

↳ $I(X^n; Y^n) = H(Y^n) - H(Y^n | X^n) = H(Y^n) - \sum_{i=1}^n H(Y_i | Y_{1:n-1}, X^n) = H(Y^n) - \sum_{i=1}^n H(Y_i | X_i)$

$$\leq \sum_{i=1}^n H(Y_i) - \sum_{i=1}^n H(Y_i | X_i) = \sum_{i=1}^n I(X_i; Y_i) \leq nC \rightarrow I(X; Y) \leq C$$

where $H(Y_i | Y_{1:n-1}, X^n) = H(Y_i | X_i)$ for a DMC,

$H(Y^n) = \sum_{i=1}^n H(Y_i | Y_{1:n-1}, X^n) \leq \sum_{i=1}^n H(Y_i)$ as conditioning can only reduce entropy

$I(X; Y) \leq C$ by defn of capacity C .

For Personal Use Only -bkwk2

- Since we assumed a uniform prior on W , $H(W) = \log |W| = \log 2^{nR} = nR$.

$$nR = H(W) = H(W|\hat{W}) + I(W;\hat{W}) = 1 + Pe nR + I(W;\hat{W}) \leq 1 + Pe nR + I(\hat{X};\hat{Y}) \leq 1 + Pe nR + nC$$

where we have shown $H(W|\hat{W}) = 1 + Pe nR$ and $I(\hat{X};\hat{Y}) \leq nC$ earlier,

and applying the data-processing inequality gives $I(W;\hat{W}) \leq I(\hat{X};\hat{Y})$ [Note $W - \hat{X} - \hat{Y} - \hat{W}$]

- This implies $Pe \geq 1 - \frac{C}{R} - \frac{1}{nR}$, thus unless $R \leq C$, Pe is bounded away from 0 as $n \rightarrow \infty$.

↳ e.g. $R = 1.1C$ and $n \rightarrow \infty$, we have $Pe \geq 1 - \frac{C}{1.1C} = 0.0909$, i.e. Pe cannot go arbitrarily small.

⇒ Second half of the channel coding thm.

+ The channel coding thm. does not give a practical technique to communicate reliably at any rate $R < C$, because

↳ Joint typical decoding is too complex to be feasible as we need to go through the codebook of size 2^{nR} . [high time complexity]

↳ The $2^{nR} \times n$ codebook is too large to store [high space complexity]

Binary linear block codes

Block codes:

- An (n, k) binary block code maps every block of k data bits into a n -length binary codeword.

↳ We want redundancy, so $n > k$

↳ The rate $R = k/n$

↳ The no. of codewords $M = 2^k$

- A good block code should have

↳ Rate $R = k/n$ as high as possible

↳ Low probability of decoding error P_e .

↳ computationally efficient ways to encode and decode.

- The two "extremes" of block codes are $(n, 1)$ repetition code and $(k+1, k)$ single parity-check code

① $(n, 1)$ repetition code, where n is odd, $R = 1/n$

↳ Encoder: Simply repeat the data bit n times

↳ Decoder: Majority vote → can reliably correct up to $\lfloor \frac{n}{2} \rfloor$ errors

② $(k+1, k)$ single parity check code, $R = k/(k+1)$

↳ Encoder: Append the modulo-two sum of the k data bits as the parity bit.

↳ Decoder: Take first k bits → can only detect one error and cannot correct it.

Minimum distance of a code

- The Hamming distance $d(x, y)$ between two binary sequences x, y of length n is the no. of positions in which x and y differ.

- Let B be a code w/ codewords $\{c_1, c_2, \dots, c_n\}$. Then the min. distance $d_{\min}$ is the smallest Hamming distance between any pair of codewords.

$$d_{\min} = \min_{i \neq j} d(c_i, c_j)$$

Optimal decoding of a block code

- Given a block code B w/ codewords $\{c_1, c_2, \dots, c_n\}$, the optimal decoder is the one that minimises the probability of decoding error P_e .

- For any channel described by $P_{Y|X}$, if the $1/p$ bits/messages corresponding to the codewords are equally likely, then the optimal decoder given the received length- n sequence y is

$$\hat{c} = \underset{c \in \{c_1, c_2, \dots, c_n\}}{\operatorname{argmax}} P(y|c) \quad [\text{max-likelihood decoder}]$$

For Personal Use Only -bkwk2

Optimal decoding on the BSC(p).

- Given that codeword $\mathbf{c}$ was transmitted and $\mathbf{y}$ was received, the no. of channel errors is $d(\mathbf{y}, \mathbf{c})$ and the no. of correctly received bits is $n - d(\mathbf{y}, \mathbf{c})$. Therefore,

$$P(\mathbf{y}|\mathbf{c}) = p^{d(\mathbf{y}, \mathbf{c})} (1-p)^{n-d(\mathbf{y}, \mathbf{c})} = (1-p)^n \left(\frac{p}{1-p}\right)^{d(\mathbf{y}, \mathbf{c})}$$

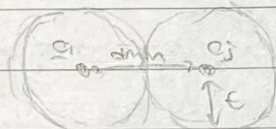
Thus, for $p < 1/2$, we have $(\frac{p}{1-p}) < 1$, and the optimal decoding rule is

$$\hat{\mathbf{c}} = \underset{\mathbf{c} \in \{c_1, \dots, c_M\}}{\operatorname{argmin}} d(\mathbf{y}, \mathbf{c})$$

i.e. choose the codeword closest in Hamming distance to $\mathbf{y}$.

* For $p > 1/2$, we can simply flip the o/p to get $p_{\text{eff}} = 1-p < 1/2$.

- consider spheres of radius t centred on each codeword.



sphere centred at $\mathbf{c}_i$ contains all the binary sequences $\mathbf{z}$ s.t. $d(\mathbf{c}_i, \mathbf{z}) \leq t$.

- If fewer than t errors occur, the received word $\mathbf{z}$ will lie within the radius- t sphere of the transmitted codeword.

- The spheres won't overlap if $2t < d_{\min}$ or $2t \leq d_{\min} - 1$, so we can correct any pattern of t errors if

we can detect any patterns of t errors
for $s < d_{\min}$

$$t \leq \left\lfloor \frac{d_{\min} - 1}{2} \right\rfloor$$

← floor since $t \in \mathbb{Z}^+$.

Linear block codes (LBC) and generator matrix $\mathbf{G}$.

- A (n, k) LBC is defined in terms of k length- n binary vectors, say $\mathbf{g}_1, \dots, \mathbf{g}_k$. A sequence of k data bits, say $\mathbf{x} = (x_1, \dots, x_k)$ is mapped to a length- n codeword $\mathbf{c}$ as follows.

$$\mathbf{c} = x_1 \mathbf{g}_1 + x_2 \mathbf{g}_2 + \dots + x_k \mathbf{g}_k$$

or alternatively,

$$\mathbf{c} = \mathbf{x} \mathbf{G} \quad \text{where} \quad \mathbf{G} = \begin{bmatrix} \mathbf{g}_1 \\ \mathbf{g}_2 \\ \vdots \\ \mathbf{g}_k \end{bmatrix}$$

The $k \times n$ matrix $\mathbf{G}$ is the generator matrix. k is the code dimension and n is the block length.

* Matrix multiplication is over the binary field, (for $x, y \in \{0, 1\}$, $x+x=0$, $x+\bar{x}=1$, $x \cdot 0=0$, $x \cdot 1=x$)
the vectors $\mathbf{c}, \mathbf{x}, \mathbf{g}_i$ are row vectors.

- Let $\mathcal{C}$ be a (n, k) LBC w/ codewords $\{\mathbf{c}_0, \dots, \mathbf{c}_{M-1}\}$. $\mathcal{C}$ is a k -dimensional subspace of $\{0, 1\}^n$ i.e.

→ 1) For any codewords $\mathbf{c}_i, \mathbf{c}_j \in \mathcal{C}$, $\mathbf{c}_i + \mathbf{c}_j$ is also in $\mathcal{C}$. [closed under vector addition]

Let $\mathbf{c}_i = a_1 \mathbf{g}_1 + \dots + a_k \mathbf{g}_k$ and $\mathbf{c}_j = b_1 \mathbf{g}_1 + \dots + b_k \mathbf{g}_k$, then $\mathbf{c}_i + \mathbf{c}_j = (a_1 + b_1) \mathbf{g}_1 + \dots + (a_k + b_k) \mathbf{g}_k$.

$\mathbf{c}_i + \mathbf{c}_j$ is just the codeword corresponding to data sequence $\mathbf{x} = (a_1 + b_1, \dots, a_k + b_k)$, $(a_i + b_i) \in \{0, 1\} \forall i$

→ 2) If $\mathbf{c} \in \mathcal{C}$, then $0 \cdot \mathbf{c}$ and $1 \cdot \mathbf{c}$ are in $\mathcal{C}$ [closed under scalar multiplication]

For $\mathbf{c} \in \mathcal{C}$, $\mathbf{c} + \mathbf{c} = 0 \rightarrow 0 \in \mathcal{C}$ // $0 \cdot \mathbf{c} = 0 \in \mathcal{C}$ and $1 \cdot \mathbf{c} = \mathbf{c} \in \mathcal{C}$.

- concepts like basis, dimension, orthogonality familiar in $\mathbb{R}^n$ can be carried over to $\{0, 1\}^n$.

→ vectors $\mathbf{u}, \mathbf{v} \in \{0, 1\}^n$ are orthogonal if $\mathbf{u} \cdot \mathbf{v}^T = 0$ (Note $\mathbf{u} \cdot \mathbf{u}^T = 0$, so not identical to $\mathbb{R}^n$)

For Personal Use Only -bkwk2

Systematic generator matrix

different G can yield the same set of codewords C , but have different mappings.

- The generator matrix for a code is not unique. Among all possible generator matrices for a code, we often prefer systematic generator matrices, which have the form

$$\underline{G} = [\underline{I}_k | \underline{P}]$$

where $\underline{I}_k$ is the $k \times k$ identity matrix and $\underline{P}$ is a $k \times (n-k)$ matrix.

- For a systematic $\underline{G} = [\underline{I}_k | \underline{P}]$, the codeword corresponding to length- k data vector $\underline{x}$ is

$$\underline{c} = \underline{x} \underline{G} = \underline{x} [\underline{I}_k | \underline{P}] = [\underline{x} | \underline{x} \underline{P}]$$

i.e. the length- n codeword consists of the k data bits $\underline{x}$, followed by $(n-k)$ parity bits $\underline{x} \underline{P}$

- Given any generator matrix, we can find a systematic version:

→ use elementary row operations - swap rows or replace any row by a sum of that row and another row.

→ same set of codewords, only the mapping is different

→ swap the columns → rearranging the components of the codewords.

Parity check matrix $\underline{H}$.

- The orthogonal complement of C , $C^\perp$ is defined as the set of all vectors in $\{0,1\}^n$ that are orthogonal to each vector in C .

- $C^\perp$ is a $(n-k)$ -dimensional subspace of $\{0,1\}^n$.

→ 1) For any position vector $\underline{h}_i, \underline{h}_j \in C^\perp$, $\underline{h}_i + \underline{h}_j \in C^\perp$ [closed under vector addition]

By def'n, for any $\underline{c} \in C$, $\underline{c} \underline{h}_i^T = 0$ and $\underline{c} \underline{h}_j^T = 0 \rightarrow \underline{c} (\underline{h}_i + \underline{h}_j)^T = \underline{c} \underline{h}_i^T + \underline{c} \underline{h}_j^T = 0 \rightarrow (\underline{h}_i + \underline{h}_j) \in C^\perp$

→ 2) If $\underline{h} \in C^\perp$, then $0 \cdot \underline{h}$ and $1 \cdot \underline{h}$ are in $C^\perp$ [closed under scalar multiplication]

For $\underline{h} \in C^\perp$, $\underline{h} + \underline{h} = 0 \rightarrow 0 \in C^\perp$ // $0 \cdot \underline{h} = 0 \in C^\perp$ and $1 \cdot \underline{h} = \underline{h} \in C^\perp$.

- we can thus find a basis $\{\underline{h}_1, \dots, \underline{h}_{n-k}\}$ for $C^\perp$, expressed as

$$\underline{H} = \begin{bmatrix} \leftarrow \underline{h}_1 \rightarrow \\ \leftarrow \underline{h}_2 \rightarrow \\ \leftarrow \vdots \rightarrow \\ \leftarrow \underline{h}_{n-k} \rightarrow \end{bmatrix}$$

The $(n-k) \times n$ matrix $\underline{H}$ is the parity check matrix.

- since every codeword $\underline{c} \in C$ is orthogonal to every row of $\underline{H}$, i.e. $\underline{c} \underline{H}^T = \underline{0}$, and $\underline{c} = \underline{x} \underline{G}$,

$$\therefore \underline{x} \underline{G} \underline{H}^T = \underline{0} \quad \forall \underline{x} \rightarrow \underline{G} \underline{H}^T = \underline{0}$$

for general G , convert to G_{sys} , then find H .

- Given the systematic $\underline{G} = [\underline{I}_k | \underline{P}]$, we can find $\underline{H}$ using $\underline{H} = [\underline{P}^T | \underline{I}_{n-k}]$ (Note $\underline{P}^T$ is $(n-k) \times k$)

→ check $\underline{G} \underline{H}^T = [\underline{I}_k | \underline{P}] \begin{bmatrix} \underline{P} \\ \underline{I}_{n-k} \end{bmatrix} = \underline{P} + \underline{P} = \underline{0}$ ($\underline{P} + \underline{P} = \underline{0}$ in $\{0,1\}^n$).

- If we expand $\underline{c} \underline{H}^T = \underline{0}$, we get k eqns, known as the parity check eqns of the code.

For Personal Use Only -bkwk2

Minimum distance of LBC. ← useful for finding t, the no. of errors it can correct up to

- The Hamming distance btwn two binary vectors u, v can be expressed as

$$d(u, v) = wt(u+v)$$

where wt refers the Hamming weight of the vector, i.e. the no. of 1s in it.

- Since the min. distance of a code is $d_{min} = \min_{i \neq j} d(c_i, c_j) = \min_{i \neq j} wt(c_i + c_j)$, and for any two codewords c_i, c_j of an LBC, $c_k = c_i + c_j$ is also a codeword,

$$d_{min} = \min_{c_k \neq 0} wt(c_k)$$

i.e. the min. distance of a LBC equals the min. Hamming weight among the non-zero codewords

- Let C be a LBC w/ parity check matrix H . The min. distance of C , d , is the smallest no. of columns that sum to 0 .

1) The sum of any $(d-1)$ or fewer columns of H is never equal to 0 .

Assume that only t columns of H sum to 0 , where $t \leq d-1$. Let these columns be $i_1 \dots i_t$.

Consider a n -length vector g which has 1s in positions $i_1 \dots i_t$ and 0 everywhere else.

$gH^T = 0$ by construction $\rightarrow g$ is a codeword and has weight $t \leq d-1 \rightarrow$ contradiction as $wt(s) \geq d$.

$\rightarrow$ 2) There is at least one set of d columns that add up to 0 .

Since d is the min. distance, there exist a codeword w/ weight d , say c , $cH^T = 0$.

Let $i_1 \dots i_d$ be the positions of the 1s in c . Then gh^T is the sum of columns $i_1 \dots i_d$ of H , which sum to 0 .

Performance of block codes over a BSC(p)

- For the BSC optimal decoder, error patterns of weight up to $\lfloor (d_{min}-1)/2 \rfloor$ will be corrected.

The probability of correctly decoding the codeword is therefore

$$P_{\text{correct}} \geq \sum_{w=0}^{\lfloor (d_{min}-1)/2 \rfloor} \binom{n}{w} p^w (1-p)^{n-w}$$

i.e. the probability that the channel flips $\lfloor (d_{min}-1)/2 \rfloor$ or fewer bits

$$\therefore P_e \leq 1 - \sum_{w=0}^{\lfloor (d_{min}-1)/2 \rfloor} \binom{n}{w} p^w (1-p)^{n-w}$$

- The above bound P_e may be pessimistic because d_{min} only gives the guaranteed error

correction capability of a code, but the code may be able to correct many more error patterns.

(many error patterns of weight $t > \lfloor (d_{min}-1)/2 \rfloor$ can be corrected, but we cannot guarantee all can be corrected)

- If we only focus on constructing codes w/ large d_{min} , we get codes w/ rates $R \ll$ capacity C .

The key is to construct codes that correct most of the typical error patterns (ones w/ weight $\approx p$)

For Personal Use Only -bkwk2

Low density parity check (LDPC) codes for the BEC

LDPCs for the BEC.

- To recover the erasures of the received sequence y , yielded from transmitting a codeword generated using a LDPC w/ generator matrix G and parity check matrix H , we solve one of the following:

$$xG = 0$$

$$yH^T = 0$$

- Considering only the columns of G , x corresponding to the non-erased values of y .

$$xG = 0 \rightarrow xG_{\text{non-erased}} = y_E \rightarrow x = y_E G_{\text{non-erased}}^{-1}$$

- Considering only the columns of G , x corresponding to the erased values of y , by making the contributions of the other columns to the binary dot product to the RHS,

$$y_E H^T = 0 \rightarrow y_E H^T = y_{\text{non-erased}} H^T \rightarrow y_E = y_{\text{non-erased}} (H^T)^{-1}$$

- e.g. consider the generator matrix G and parity check matrix H .

$$G = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 1 & 1 & 0 & 1 \end{bmatrix} \quad H = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 & 1 \end{bmatrix}$$

Block length $n=7$, dimension $k=3$, rate $= k/n = 3/7$.

A codeword generated using G is transmitted over a BEC and the channel output is $y = [1 \ a \ b \ 0 \ c \ 1 \ d]$

(i) Use $xG = 0$. Considering the non-erased columns, $G_{\text{non-erased}} = \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$, $y_E = [1 \ 0 \ 1]$,

$$x \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix} = [1 \ 0 \ 1] \rightarrow x = [1 \ 0 \ 1]$$

$$\therefore e = xG = [1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1]$$

(ii) Use $y_E H^T = 0$. Considering the erased/non-erased columns, $H_E = \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix}$, $y_E = [0 \ b \ c \ d]$,

$$H_{\text{non-erased}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, y_{\text{non-erased}} = [1 \ 0 \ 1] \rightarrow y_E H_E^T = [1 \ 0 \ 1] \begin{bmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} = [1 \ 1 \ 0 \ 0]$$

$$y_E \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = [1 \ 1 \ 0 \ 0] \rightarrow y_E = [0 \ 1 \ 0 \ 1]$$

$$\therefore e = [1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1]$$

- Consider using the $y_E H^T = 0$ method. For a $(n-k) \times n$ parity check matrix H , and a received seq. y w/ e erasures, moving the $y_{\text{non-erased}} H^T$ term to RHS gives $(n-k)$ linear eqns. These can be solved if the no. of erasures $e \leq$ the no. of independent rows of H_E .

- The above matrix-inversion decoder is optimal for the BEC, i.e. if the pattern of erasures cannot be solved by this decoder, other decoders cannot solve it either.

- The matrix-inversion decoder scales $O(n^2)$ w/ block length n (due to Gaussian elimination)

For Personal Use Only -bkwk2

Iterative decoding

- By considering opportunistic back substitution, we get the faster iterative decoder.
- We look for a row in the parity check matrix H which has exactly one 1 among the erased locations, then update x accordingly by solving the linear eqn. corresponding to that row.

- e.g. Decode $y = [1 \ a \ b \ 0 \ c \ 1 \ d]$ yielded from a codebook generated from the LSC w/ parity

check matrix $H = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$, transmitted over a BSC, using an iterative decoder.

$$(i) \quad H = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$(ii) \quad H = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$(iii) \quad H = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$(iv) \quad H = \begin{bmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$y = [1 \ a \ b \ 0 \ c \ 1 \ d]$$

$$y = [1 \ 0 \ b \ 0 \ c \ 1 \ d]$$

$$y = [1 \ 0 \ 1 \ 0 \ c \ 1 \ d]$$

$$y = [1 \ 0 \ 1 \ 0 \ 0 \ 1 \ d]$$

$$\therefore y h_1^T = 0 \rightarrow a = 0$$

$$\therefore y h_2^T = 0 \rightarrow b = 1$$

$$\therefore y h_3^T = 0 \rightarrow c = 0$$

$$\therefore y h_4^T = 0 \rightarrow d = 1$$

$$\rightarrow c = [1 \ 0 \ 1 \ 0 \ 0 \ 1 \ 1]$$

- The iterative decoder is not an optimal decoder - it won't always work, even when the received sequence y has an error pattern decodable by other decoders.
- The iterative decoder is possible when the effective matrix $y H^T$ can be triangulated by just row swaps $\rightarrow$ this req. the parity check matrix H to have a low density of 1s.

Low density parity check (LDPC) matrix.

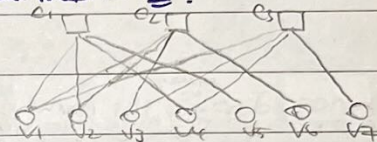
- The weight of a row d_r / column d_c is the no. of 1s in the row / column.
- The average of the $(n-k)$ row weights / n column weights are denoted by $\bar{d}_r / \bar{d}_c$ respectively.
- A regular LDPC matrix has row weight d_r for all $(n-k)$ rows and column weight d_c for all n columns. $\rightarrow$ it suffices to specify d_r and d_c .
- An irregular LDPC matrix does not have the same row weight d_r / column weight d_c for all rows / columns. $\rightarrow$ we need to specify the weight distributions on the rows / columns.
- A row w/ weight d_r is a parity check eqn. that involves d_r code bits ("c" $\rightarrow$ check)
- A column w/ weight d_c is a codeword bit involved in d_c parity check eqns ("v" $\rightarrow$ variable)
- The no. of 1s in a (n, k) LDPC matrix is given by
 - $\rightarrow$ Regular LDPC matrix : $n d_c = (n-k) d_r \rightarrow \frac{k}{n} = 1 - \frac{d_r}{d_c}$
 - $\rightarrow$ Irregular LDPC matrix : $n \bar{d}_c = (n-k) \bar{d}_r \rightarrow \frac{k}{n} = 1 - \frac{\bar{d}_r}{\bar{d}_c}$
- $\frac{k}{n}$ is the design rate, which is the true code rate if the rows of the parity check matrix are linearly independent (or we can remove the redundant rows $\rightarrow$ same n , smaller $n-k$ $\rightarrow$ larger k , $\frac{k}{n} \therefore R_{true} \geq R_{design}$).

For Personal Use Only -bkwk2

Factor graph of a linear code.

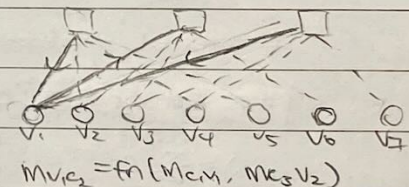
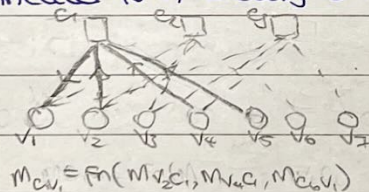
- We can represent the parity check matrix H as a factor graph to analyse the iterative decoder and design good LDPC matrices (both are equal representations).
- Each column j of H is a variable node v_j (circle).
- Each row i of H is a check node c_i (square).
- A 1 in entry (i,j) means that variable node j is connected to check node i .
- e.g. Draw the factor graph of the parity check matrix H .

$$H = \begin{bmatrix} 1 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \begin{matrix} c_1 \\ c_2 \\ c_3 \end{matrix}$$



Iterative decoding as message passing.

- Message passing is a class of iterative algorithms, where in each step:
 - 1) Each variable node v sends a message m_{vc} to each check node c that it is connected to. The message m_{vc} is a function of the messages $\{m_{v'c}\}$ coming in from variable nodes v' connected to c , excluding v itself, and the channel o/p.
 - 2) Each check node c sends a message m_{cv} to each variable node v that it is connected to. The message m_{cv} is a function of the messages $\{m_{c'v}\}$ coming in from check nodes c' connected to v , excluding c itself.



- The message passing rules for decoding on the BEC are as follows:

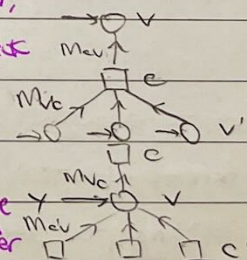
- In the first step, m_{vc} is the channel o/p corresponding to bit v (0, 1 or ?)
- In each subsequent step,

$$m_{cv} = \begin{cases} \sum_v m_{vc} \bmod 2 & \text{if no } m_{vc} = ? \\ ? & \text{if at least one incoming } m_{vc} = ? \end{cases}$$

$$m_{vc} = \begin{cases} x \in \{0, 1\} & \text{if } v = x \text{ or at least one incoming } m_{cv} = x \\ ? & \text{if } v = ? \text{ and all incoming } m_{cv} = ? \end{cases}$$

only 1 unknown v , can be solved using parity check

v has non-unique or got deleted earlier



- In summary, in each step of the message passing decoder,

- m_{vc} : variable node v tells check node c its best guess of its own value
- m_{cv} : check node c tells variable node v what it thinks the value of v should be (decodes it)

- The complexity of the message passing decoder $\propto$ (# of edges in graph) (# of iterations)

i.e. low density of 1s in $H \rightarrow$ low complexity of decoder.

LDPC or turbo codes

rule for other channel decoders too!

For Personal Use Only -bkwk2

Degree distributions

- The degree distribution specifies the distribution of weights on the nodes/edges of the graph.

- We can define the degree distribution using the node perspective polynomials

$$L(x) = \sum_{i=1}^{\text{degmax}} L_i x^i$$

$$R(x) = \sum_{i=1}^{\text{degmax}} R_i x^i$$

← coeffs sum to 1

where L_i is the fraction of left (variable) nodes of degree i (fraction of columns in $\mathbb{H}$ w/ weight i)

R_i is the fraction of right (check) nodes of degree i (fraction of rows in $\mathbb{H}$ w/ weight i)

- The average degree of a variable / check node satisfy

$$\bar{d}_v = L'(1)$$

$$\bar{d}_c = R'(1)$$

$$\hookrightarrow L(x) = \sum_{i=1}^{\text{degmax}} L_i x^i \rightarrow L'(x) = \sum_{i=1}^{\text{degmax}} i L_i x^{i-1}, \quad L'(1) = \sum_{i=1}^{\text{degmax}} i L_i = \bar{d}_v$$

$$\hookrightarrow R(x) = \sum_{i=1}^{\text{degmax}} R_i x^i \rightarrow R'(x) = \sum_{i=1}^{\text{degmax}} i R_i x^{i-1}, \quad R'(1) = \sum_{i=1}^{\text{degmax}} i R_i = \bar{d}_c$$

$$\hookrightarrow \text{The design rate } r \text{ thus } k/n = 1 - \frac{\bar{d}_v}{\bar{d}_c} = 1 - \frac{L'(1)}{R'(1)}$$

- We can define the degree distribution using the edge perspective polynomials.

$$\lambda(x) = \sum_{i=1}^{\text{degmax}} \lambda_i x^{i-1}$$

$$\rho(x) = \sum_{i=1}^{\text{degmax}} \rho_i x^{i-1}$$

← coeffs sum to 1.

where λ_i is the fraction of edges connected to variable nodes of degree i (fraction of 1s in $\mathbb{H}$ in columns w/ weight i)

ρ_i is the fraction of edges connected to check nodes of degree i (fraction of 1s in $\mathbb{H}$ in rows w/ weight i)

- The average degree of a variable / check node satisfy

$$\bar{d}_v = \left(\int_0^1 \lambda(x) dx \right)^{-1}$$

$$\bar{d}_c = \left(\int_0^1 \rho(x) dx \right)^{-1}$$

$$\hookrightarrow \lambda(x) = \sum_{i=1}^{\text{degmax}} \lambda_i x^{i-1} \rightarrow \int_0^1 \lambda(x) dx = \left[\sum_{i=1}^{\text{degmax}} \frac{\lambda_i}{i} x^i \right]_0^1 = \sum_{i=1}^{\text{degmax}} \frac{\lambda_i}{i}$$

edges = L , # edges connected to variable node w/ wt $i = L \lambda_i$, # variable nodes w/ wt $i = \frac{L \lambda_i}{i}$

$$\# \text{ variable nodes} = \sum_{i=1}^{\text{degmax}} \frac{L \lambda_i}{i} = n \rightarrow \bar{d}_v = \frac{L}{n} = \left(\sum_{i=1}^{\text{degmax}} \frac{\lambda_i}{i} \right)^{-1} = \left(\int_0^1 \lambda(x) dx \right)^{-1}$$

$$\hookrightarrow \rho(x) = \sum_{i=1}^{\text{degmax}} \rho_i x^{i-1} \rightarrow \int_0^1 \rho(x) dx = \left[\sum_{i=1}^{\text{degmax}} \frac{\rho_i}{i} x^i \right]_0^1 = \sum_{i=1}^{\text{degmax}} \frac{\rho_i}{i}$$

edges = L , # edges connected to check node w/ wt $i = L \rho_i$, # check nodes w/ wt $i = \frac{L \rho_i}{i}$

$$\# \text{ check nodes} = \sum_{i=1}^{\text{degmax}} \frac{L \rho_i}{i} = n - k \rightarrow \bar{d}_c = \frac{L}{n - k} = \left(\sum_{i=1}^{\text{degmax}} \frac{\rho_i}{i} \right)^{-1} = \left(\int_0^1 \rho(x) dx \right)^{-1}$$

$$\hookrightarrow \text{The design rate is thus } k/n = 1 - \frac{\bar{d}_v}{\bar{d}_c} = 1 - \frac{\int_0^1 \lambda(x) dx}{\int_0^1 \rho(x) dx}$$

Density evolution.

- Density evolution is a technique to predict the decoding performance of codes w/ a given $\lambda(x), \rho(x)$ for large n . ($\lambda(x), \rho(x)$ already determine the rate k/n , so we want to find ϵ^{HP}).

- Density evolution recursion predicts the fraction of erased bits at the end of each step t , P_t

We can find the max. $P_0 = \epsilon$ for which $P_t \rightarrow 0$ as $t \rightarrow \infty$, which is denoted by ϵ^{HP} ← find by trial and error

- A key assumption of density evolution is that the incoming messages at each node are independent.

↳ This is strictly true if there are no cycles/loops in the factor graph (i.e. a tree)

↳ A practical LDPC code is almost never cycle-free, but the independence assumption is close enough for large n as long as the factor graph doesn't have short cycles.

For Personal Use Only -bkwk2

- Let P_e denote the probability that an outgoing $u \rightarrow v$ message is an erasure in step t .

On a BEC w/ erasure probability ϵ , for $t=0$, we have $P_0 = \epsilon$.

- Let q_e denote the probability that an outgoing $v \rightarrow u$ message is an erasure in step t .

We start w/ $q_0 = 1$ as the first $v \rightarrow u$ messages are all erasures.

- For $t \geq 1$, assuming that all the incoming messages are independent, we have:

↳ ① Regular LDPC codes. (Given d_v, d_c).

Probability that the variable AND all the $d_v - 1$ incoming messages are erasures is

$$P_e = \epsilon (q_{t-1})^{d_v - 1}$$

Probability that at least one of the $d_c - 1$ incoming messages is an erasure is

$$q_t = 1 - (1 - P_e)^{d_c - 1}$$

Combining the two eqns, we get the recurrence relation

$$P_e = \epsilon (1 - (1 - P_e)^{d_c - 1})^{d_v - 1}$$

↳ ② Irregular LDPC codes (Given $\lambda(x), \rho(x)$)

Probability that an outgoing message from a variable node w/ deg. i is erased is $P_{e,i} = \epsilon (q_{t-1})^{i-1}$

$\therefore$ Probability that a randomly picked edge has $M_{v,c} = ?$ is

$$P_e = \sum_i \lambda_i P_{e,i} = \epsilon \sum_i \lambda_i (q_{t-1})^{i-1} = \epsilon \lambda(q_{t-1})$$

Probability that an outgoing message from a check node w/ deg. j is erased is $q_{t,j} = 1 - (1 - P_e)^{j-1}$

$\therefore$ Probability that a randomly picked edge has $M_{c,v} = ?$ is

$$q_t = \sum_j \rho_j q_{t,j} = 1 - \sum_j \rho_j (1 - P_e)^{j-1} = 1 - \rho(1 - P_e)$$

Combining the two eqns, we get the recurrence relation

$$P_e = \epsilon \lambda(1 - \rho(1 - P_e))$$

- Irregular LDPC ensembles give us more flexibility in the code design and can be optimised to get the max. possible threshold ϵ^{LP} closer to the Shannon limit $\epsilon^* = 1 - R$ for a given rate R .

- We would need to optimise over $\lambda(x), \rho(x)$, which is a non-trivial optimisation problem.

For example, for $R=0.5$, it is found the following degree distribution has ϵ^{LP} close to $\epsilon^* = 0.5$.

$$\lambda(x) = 0.4300x^2 + 0.2773x^3 + 0.0080x^4 + 0.1195x^{47} + 0.0522x^{48} + 0.0796x^{61} + 0.0734x^{62}$$

$$\rho(x) = 0.7138x^9 + 0.1225x^{10} + 0.1637x^{19}.$$

For Personal Use Only -bkwk2

Low density parity check (LDPC) codes for general binary input channels

The setup.

- We will consider the BSC(p) and B-AWGN channel.
- The B-AWGN channel is $Y = X + N$, where the i/p $X \in \{+1, -1\}$ and $N \sim \mathcal{N}(0, \sigma^2)$ is additive white Gaussian noise.
- Using a LDPC code, we can map our information bits to a codeword $s = (c_1, \dots, c_n)$. The codeword s is then transmitted over the channel. (Note for B-AWGN channel we map $c_i = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \leftrightarrow X_i = \begin{bmatrix} +1 \\ -1 \end{bmatrix}$)
- The decoder receives $y = (y_1, \dots, y_n)$ and has to decide s .

Optimal decoding: block-wise vs bit-wise.

- For a linear code C , the optimal decoding rule to minimise the probability of block (codeword) decoding error is [Blockwise optimal decoding rule]

$$\hat{C} = \underset{C \in C}{\operatorname{argmax}} P(C|y)$$

provided the codewords are equally likely, i.e. $P(C)$ is const, then using Bayes rule, $P(C|y) = \frac{P(C)P(y|C)}{P(y)}$

$$\underset{C \in C}{\operatorname{argmax}} P(C|y) = \underset{C \in C}{\operatorname{argmax}} P(y|C) \rightarrow \hat{C} = \underset{C \in C}{\operatorname{argmax}} P(y|C)$$

- For the BSC(p), the blockwise optimal decoding rule becomes.

$$\hat{C} = \underset{C \in C}{\operatorname{argmax}} P(y|C) = \underset{C \in C}{\operatorname{argmin}} d(y, C)$$

→ Finding the codeword closest to y , among 2^{nR} codewords, is hard.

- For a linear code C , the optimal decoding rule to minimise the probability of bit decoding error is [bit-wise optimal decoding rule]

$$\hat{C}_j = \underset{C_j \in \{0,1\}}{\operatorname{argmax}} P(C_j|y) \quad \text{for } j=1, \dots, n.$$

- For the BSC(p), for each $j=1, \dots, n$, the bit-wise optimal decoding rule becomes.

$$\hat{C}_j = \underset{C_j \in \{0,1\}}{\operatorname{argmax}} P(C_j|y) = \underset{b \in \{0,1\}}{\operatorname{argmax}} \sum_{C: C_j=b} \left(\frac{P}{1-P}\right)^{d(y, C)}$$

→ consider $P(C_j=0|y)$, and using Bayes rule, we can write

$$\begin{aligned} P(C_j=b|y) &= \frac{P(C_j=b, y)}{P(y)} = \frac{1}{P(y)} \sum_{C: C_j=b} P(C|y) = \frac{1}{P(y)} \sum_{C: C_j=b} P(C)P(y|C) = \frac{1}{P(y)} \sum_{C: C_j=b} P(C)P^{d(y, C)} \left(\frac{P}{1-P}\right)^{n-d(y, C)} \\ &= \frac{2^{nR} P^{nR}}{P(y)} \sum_{C: C_j=b} \left(\frac{P}{1-P}\right)^{d(y, C)} \rightarrow \underset{C_j \in \{0,1\}}{\operatorname{argmax}} P(C_j|y) = \underset{b \in \{0,1\}}{\operatorname{argmax}} \sum_{C: C_j=b} \left(\frac{P}{1-P}\right)^{d(y, C)} \end{aligned}$$

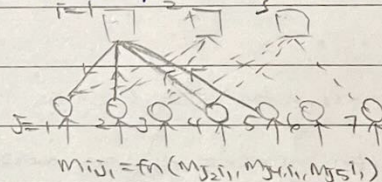
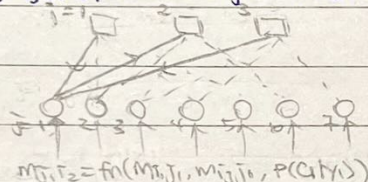
→ Hard to implement - for each bit j , it req. computing the complicated quantity $\sum_{C: C_j=b} \left(\frac{P}{1-P}\right)^{d(y, C)}$ for both $b=0$ and $b=1$, then compare the two sums.

- * For bit-wise decoding, if we want to compute $P(C_j=b|y)$, consider $\frac{P(C_j=b|y)}{P(C_j=1|y)}$ (to cancel out the unknown $P(y)$) and $P(C_j=0|y) + P(C_j=1|y) = 1$

For Personal Use Only -bkwk2

Message passing (MP) decoding

- We will focus on a MP algorithm that approximately computes the desired bit-wise a posteriori probabilities (APPs) $P(g_j=0|y_j)$ [sum-product algorithm / belief propagation (BP)]
- We will index the variable nodes by j , $j=1 \dots n$ and check nodes by i , $i=1 \dots m$.
- In each iteration, the MP decoder computes:
 - ↳ Each variable node j sends a message m_{ji} to each check node i that it is connected to. m_{ji} is a function of the incoming messages into j except from check node i itself, and the channel evidence $P(g_j|y_j)$
 - ↳ Each check node i sends a message m_{ij} to each variable node j that it is connected to. m_{ij} is a function of the incoming messages into i except from variable node j itself.
- $\underline{m}_{ji} = \begin{bmatrix} m_{ji}(0) \\ m_{ji}(1) \end{bmatrix}$, where $m_{ji}(b)$ is an updated estimate of the APP that $g_j=b$.
- $\underline{m}_{ij} = \begin{bmatrix} m_{ij}(0) \\ m_{ij}(1) \end{bmatrix}$, where $m_{ij}(b)$ is an updated estimate that parity check eqn i is satisfied when $g_j=b$.



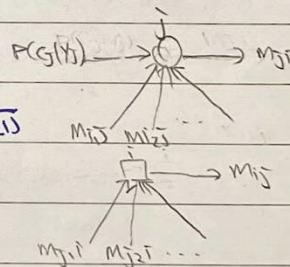
- Assuming that the incoming messages at each node are independent, the message passing rules are:

↳ In the first step, $\underline{m}_{ji} = \begin{bmatrix} m_{ji}(0) \\ m_{ji}(1) \end{bmatrix} = \begin{bmatrix} P(g_j=0|y_j) \\ P(g_j=1|y_j) \end{bmatrix}$

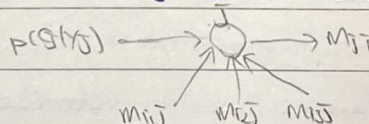
↳ In each subsequent step,

$$\underline{m}_{ji} = \begin{bmatrix} m_{ji}(0) \\ m_{ji}(1) \end{bmatrix} \quad \begin{aligned} m_{ji}(0) &= \frac{P(g_j=0|y_j) \prod_{i'} m_{ji'}(0)}{P(g_j=0|y_j) \prod_{i'} m_{ji'}(0) + P(g_j=1|y_j) \prod_{i'} m_{ji'}(1)} \\ m_{ji}(1) &= 1 - m_{ji}(0) = \frac{P(g_j=1|y_j) \prod_{i'} m_{ji'}(1)}{P(g_j=0|y_j) \prod_{i'} m_{ji'}(0) + P(g_j=1|y_j) \prod_{i'} m_{ji'}(1)} \end{aligned}$$

$$\underline{m}_{ij} = \begin{bmatrix} m_{ij}(0) \\ m_{ij}(1) \end{bmatrix} \quad \begin{aligned} m_{ij}(0) &= \frac{1}{2} + \frac{1}{2} \prod_{j'} (1 - 2m_{ji'}(1)) \\ m_{ij}(1) &= 1 - m_{ij}(0) = \frac{1}{2} - \frac{1}{2} \prod_{j'} (1 - 2m_{ji'}(1)) \end{aligned}$$



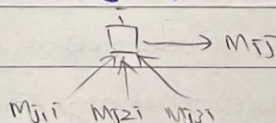
- consider the variable to check message m_{ji} for the variable node j below.



Applying Bayes rule, we get $m_{ji}(0) = \frac{a_0}{a_0 + a_1}$, $m_{ji}(1) = \frac{a_1}{a_0 + a_1}$

where $a_0 = P(g_j=b \text{ and checks } i_1, i_2, i_3 \text{ are all satisfied} | y_j) = P(g_j=b | y_j) m_{ji1}(b) m_{ji2}(b) m_{ji3}(b)$

- consider the check to variable message m_{ij} for the check node i below.



For $g_j=b$, the check eqn. is satisfied when $\begin{bmatrix} g_{i1} \\ g_{i2} \\ g_{i3} \end{bmatrix} = \begin{bmatrix} b \\ b \\ b \end{bmatrix}$ or $\begin{bmatrix} g_{i1} \\ g_{i2} \\ g_{i3} \end{bmatrix} = \begin{bmatrix} \bar{b} \\ \bar{b} \\ \bar{b} \end{bmatrix}$ or $\begin{bmatrix} g_{i1} \\ g_{i2} \\ g_{i3} \end{bmatrix} = \begin{bmatrix} b \\ \bar{b} \\ \bar{b} \end{bmatrix}$ or $\begin{bmatrix} g_{i1} \\ g_{i2} \\ g_{i3} \end{bmatrix} = \begin{bmatrix} \bar{b} \\ b \\ \bar{b} \end{bmatrix}$, so

$$m_{ij}(b) = m_{ji1}(b) m_{ji2}(b) m_{ji3}(b) + m_{ji1}(\bar{b}) m_{ji2}(\bar{b}) m_{ji3}(\bar{b}) + m_{ji1}(b) m_{ji2}(\bar{b}) m_{ji3}(\bar{b}) + m_{ji1}(\bar{b}) m_{ji2}(b) m_{ji3}(\bar{b})$$

In general, $m_{ij}(b)$ is obtained by multiplying the incoming m_{ji} s.t. on each of these are evaluated as b , and the rest as $\bar{b}$.

For Personal Use Only -bkwk2

- To obtain an expression for $M_j(0)$ in a compact form for the general case, we use the result that for a sequence of M indep. binary digits $b_1, \dots, b_M$ s.t. $P(b_k=1) = P_k \forall k$, the probability that the sequence $(b_1, \dots, b_M)$ contains an even no. of 1s is $P(\text{even}) = \frac{1}{2} + \frac{1}{2} \prod_{k=1}^M (1-2P_k)$

→ Proof by induction. Define q_1 as the prob. of having an even no. of 1s in $(b_1, \dots, b_M)$,

$M=1$, $q_1 = 1 - P_1$, the prob. the single binary digit is 0 (so even (0) no. of 1s),

$$\frac{1}{2} + \frac{1}{2} \prod_{k=1}^1 (1-2P_k) = \frac{1}{2} + \frac{1}{2} (1-2P_1) = 1 - P_1 \rightarrow \text{true for } M=1$$

Assume $q_M = \frac{1}{2} \prod_{k=1}^M (1-2P_k)$ is true for $M=r$, i.e. $q_r = \frac{1}{2} \prod_{k=1}^r (1-2P_k)$

$$M=r+1, q_{r+1} = q_r(1-P_{r+1}) + (1-q_r)P_{r+1} = \left[\frac{1}{2} + \frac{1}{2} \prod_{k=1}^r (1-2P_k) \right] (1-P_{r+1}) + \left[\frac{1}{2} - \frac{1}{2} \prod_{k=1}^r (1-2P_k) \right] P_{r+1}$$

$$= \frac{1}{2} (1 - P_{r+1} + P_{r+1}) + \left[\frac{1}{2} \prod_{k=1}^r (1-2P_k) \right] (1 - P_{r+1} - P_{r+1}) = \frac{1}{2} + \frac{1}{2} \prod_{k=1}^{r+1} (1-2P_k) \rightarrow \text{true for } M=r+1$$

→ Applying this to the check to variable message, we have $m_j(0) = \frac{1}{2} + \frac{1}{2} \prod_{i \in \mathcal{N}_j} (1-2m_i(1))$

- The complexity of the message passing decoder $\propto (\# \text{ of edges in graph}) (\# \text{ of iterations})$.

i.e. low density of 1s in $\mathbf{H} \rightarrow$ low complexity of decoder.

channel evidence $P(g|y_j)$

- Each variable to check message involves the APP $P(g|y_j)$. This can be found by inspection

or using Bayes rule $P(g=b|y_j) = \frac{P(g=b)P(y_j|g=b)}{P(y_j)} = \frac{P(g=b)P(y_j|g=b)}{P(g=b)P(y_j|g=b) + P(g=\bar{b})P(y_j|g=\bar{b})}$

- For BEC(ϵ): $y_j \in \{0, 1, ?\} \rightarrow P(g=0|y_j) = \begin{cases} 0 & \text{if } y_j=0 \\ 1/2 & \text{if } y_j=1 \\ 1/2 & \text{if } y_j=? \end{cases}$

For BSC(p): $y_j \in \{0, 1\} \rightarrow P(g=0|y_j) = \begin{cases} 1-p & \text{if } y_j=0 \\ p & \text{if } y_j=1 \end{cases}$

For B-AWGN: $y_j \in \mathbb{R} \rightarrow P(g=0|y_j) = P_{X,Y}(X_j=+1|Y_j) = \frac{P_X(X_j=+1)P_Y(+1)}{P_X(X_j=+1)P_Y(+1) + P_X(X_j=-1)P_Y(-1)}$

$$P(g=0|y_j) = \frac{\frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{(y_j-1)^2}{2\sigma^2}) \cdot \frac{1}{2}}{\frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{(y_j-1)^2}{2\sigma^2}) \cdot \frac{1}{2} + \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{(y_j+1)^2}{2\sigma^2}) \cdot \frac{1}{2}} = \frac{1}{1 + \exp(-2y_j/\sigma^2)}$$

and for all channels, $P(g=1|y_j) = 1 - P(g=0|y_j)$

Log-domain message passing.

- The BP algorithm above involves multiplying lots of probabilities $\rightarrow$ numerically unstable implementation.

→ implement w/ log-likelihood ratios (LLRs) so multiplications become additions.

- We now have the message and channel evidence in LLR form.

$$L_j = \ln \frac{M_j(0)}{M_j(1)}$$

$$L_{ij} = \ln \frac{M_{ij}(0)}{M_{ij}(1)}$$

$$L(y_j) = \ln \frac{P(g=0|y_j)}{P(g=1|y_j)} = \ln \frac{P(g=0)P(y_j|g=0)}{P(g=1)P(y_j|g=1)} = \ln \frac{P(y_j|g=0)}{P(y_j|g=1)}$$

assumed uniform prior $P(g=0) = P(g=1)$

- Assuming that the incoming messages at each node are independent, the message passing rules are:

→ In the first step, set $L_{ji} = L(y_j)$ for all edges $j \rightarrow i$ and set $L_{ji} = 0$ for all edges $i \rightarrow j$.

→ In each subsequent step,

$$L_{ji} = L(y_j) + \sum_{i' \in \mathcal{N}_j, i' \neq i} L_{ji'}$$

$$L_{ij} = 2 \tanh^{-1} \left[\prod_{j' \in \mathcal{N}_i, j' \neq j} \tanh\left(\frac{1}{2} L_{ji'}\right) \right]$$

For Personal Use Only -bkwk2

- The LLR form of the messages L_{ji}, L_{ji} can be derived as follows:

$$\rightarrow L_{ji} = \ln \frac{P(y_j=0)}{P(y_j=1)} = \ln \frac{P(y_j=0|y_i)}{P(y_j=1|y_i)} + \sum_{i' \neq i} \ln \frac{P(y_j=0|y_{i'})}{P(y_j=1|y_{i'})} = L(y_j) + \sum_{i' \neq i} L_{ji'}$$

$$\rightarrow \tanh\left(\frac{1}{2} \ln\left(\frac{1-P}{P}\right)\right) = \frac{e^{\frac{1}{2} \ln\left(\frac{1-P}{P}\right)} - 1}{e^{\frac{1}{2} \ln\left(\frac{1-P}{P}\right)} + 1} = \frac{\frac{1-P}{P} - 1}{\frac{1-P}{P} + 1} = 1 - 2P$$

$$\text{For } p = m_{ji}(1), \quad 1 - 2m_{ji}(1) = \tanh\left(\frac{1}{2} \ln\left(\frac{m_{ji}(0)}{m_{ji}(1)}\right)\right) = \tanh\left(\frac{1}{2} L_{ji}\right)$$

$$\text{For } p = m_{ji}(1), \quad 1 - 2m_{ji}(1) = \tanh\left(\frac{1}{2} \ln\left(\frac{m_{ji}(0)}{m_{ji}(1)}\right)\right) = \tanh\left(\frac{1}{2} L_{ji}\right)$$

$$1 - 2m_{ji}(1) = 1 - 2\left[\frac{1}{2} - \frac{1}{2} \prod_{j'} (1 - 2m_{ji}(1))\right] = \prod_{j'} (1 - 2m_{ji}(1)) \rightarrow \tanh\left(\frac{1}{2} L_{ji}\right) = \prod_{j'} \tanh\left(\frac{1}{2} L_{ji}\right)$$

$$\rightarrow \boxed{L_{ji} = 2 \tanh^{-1}\left[\prod_{j'} \tanh\left(\frac{1}{2} L_{ji}\right)\right]}$$

- The algorithm is run for a pre-determined no. of steps, and the final LLRs for each code bit are computed as

$$\boxed{L_j = L(y_j) + \sum_i L_{ji}} \quad \text{for } j=1 \dots n$$

where the sum in this final stage is over all the checks i connected to code bit j .

- A high +ve value of L_j indicates a high prob. that code bit j equals 0.

A highly -ve value of L_j indicates a high prob. that code bit j equals 1.

$\rightarrow$ the final decoded codeword is $\hat{c} = (\hat{c}_1, \dots, \hat{c}_n)$ where $\hat{c}_j = \begin{cases} 0 & \text{if } L_j \geq 0 \\ 1 & \text{if } L_j < 0 \end{cases}$.

- The complexity of the message passing decoder $\propto$ (# of edges in graph) (# of iterations)

i.e. low density of LDPC $\rightarrow$ low complexity of decoder.

- There are reduced complexity versions to this message passing algorithm, e.g. min-sum algorithm.

- The min-sum algorithm simplifies the check to variable message involving $\tanh$ and $\tanh^{-1}$.

$$\boxed{L_{ji} = \text{sign}(L_{ji}) |L_{ji}| = \prod_{j'} \text{sign}(L_{ji}) \frac{\min_j |L_{ji}|}{\prod_{j'} |L_{ji}|}$$

where $\text{sign}(x) \in \{+1, -1\}$ denotes the sign of x .

(this gives slightly worse performance, but much easier to implement in hardware).

channel evidence $P(y_j)$ in LLR form.

$$\text{For BEC}(\epsilon): y_j \in \{0, 1, ?\} \rightarrow \boxed{L(y_j) = \ln \frac{P(y_j=0)}{P(y_j=1)} = \begin{cases} \infty & \text{if } y_j=0 \\ -\infty & \text{if } y_j=1 \\ 0 & \text{if } y_j=? \end{cases}}$$

$$\text{For BSC}(p): y_j \in \{0, 1\} \rightarrow \boxed{L(y_j) = \ln \frac{P(y_j=0)}{P(y_j=1)} = \begin{cases} \ln \frac{1-p}{p} & \text{if } y_j=0 \\ -\ln \frac{1-p}{p} & \text{if } y_j=1 \end{cases}}$$

$$\text{For B-AWGN}: y_j \in \mathbb{R} \rightarrow \boxed{L(y_j) = \ln \frac{P(y_j=0)}{P(y_j=1)} = \ln \frac{e^{-y_j^2/\sigma^2}}{e^{-y_j^2/\sigma^2}} = \frac{2y_j}{\sigma^2}}$$

constructing good LDPC codes.

- For a given rate R , the goal is to choose $\lambda(x) p(x)$ to give the max. ϵ^{MP} (ideally close to ϵ^*).

Then pick a H matrix w/ the chosen degree distribution (randomly/deterministically).

- ϵ^{MP} is found via density evolution, but it is harder to write down and optimise for general binary input channels.